A

Project Report

on

AI Based Intrution Detection System (IDS)

Developed By

Priyansh Nagar [2505102140025]

Rohit Majhi [2505102140028]

Saksham Pandey [2505102140046]

as

Partial Fulfillment of Semester III

Master of Science in information

for A.Y. 2026 - 2027

Under the Guidance of

Dr. Chetan Gondaliya

Submitted To

Department of MScIT
Faculty of IT & Computer Science
PARUL University



CERTIFICATE

This is to certify that **Mr. Priyansh Nagar, 2505102140025, Mr. Rohit Majhi, 2505102140028**, and **Mr. Saksham Pandey, 2505102140046**, students of Master of Science in Information Technology has satisfactorily completed the Minor Project on **"AI Based Intrution Detection System"** at **Faculty of IT & Computer Science, Parul University** as partial fulfillment of M.Sc. (IT) Semester III.

Seat No. _____ Date of Submission: _____

_____	_____	_____
Internal Guide	Project Coordinator	HOD/HOI

Faculty of IT & Computer Science

PARUL University, Vadodara

PREFACE

Cybersecurity systems increasingly need to process large volumes of network telemetry and distinguish legitimate activity from traffic that may indicate malicious behaviour. Conventional rule-based monitoring remains useful, but its coverage depends on known signatures, carefully maintained rules, and the ability of Users to interpret alerts. Machine learning offers a complementary approach by learning patterns from labelled traffic records and applying those patterns to previously unseen records, subject to the quality and representativeness of the training data.

NetSentinel AI is an academic minor project that explores this approach through a web-based intrusion-detection workflow. The application allows a user to authenticate, upload a network-traffic CSV file, prepare the data for analysis, obtain model predictions, and review the resulting summary and report history. The project combines cybersecurity concepts with Python-based data processing, machine learning, web application development, and database persistence.

This report follows the chapter order and documentation style of the supplied sample report. It includes project profile, requirement analysis, system design, implementation, testing and evaluation, future scope, and bibliography. Diagrams are presented as clean, monochrome, block-oriented figures suitable for an academic report. Where implementation-specific values—such as final model metrics or exact deployment performance—were not present in the source materials, the report explicitly identifies them as values to be verified rather than inventing results.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who contributed to the successful completion of our Semester III project, **NetSentinel AI**. We are thankful to Parul University and the Faculty of Information Technology and Computer Science for providing the academic environment, facilities, and resources necessary to carry out this project successfully.

We extend our heartfelt appreciation to our Project Guide, **Dr. Chetan Gondaliya**, for his valuable guidance, continuous support, constructive feedback, and encouragement throughout the project. His expertise and suggestions played an important role in helping us achieve our project objectives.

We would like to express our sincere gratitude to the Head of the Department, **Prof. Vivek Dave**, and **Dr. Ghanshyam Rathod** for their valuable academic support and encouragement throughout the project. We extend our heartfelt appreciation to the Dean, **Dr. Priya Swaminarayan**, for her support and for fostering an enriching academic environment. We are equally grateful to the Principal, **Dr. Hina Choksi**, for providing us with the opportunity and institutional support to undertake this project as part of our academic curriculum.

We would also like to acknowledge all the faculty members and staff of the Faculty of Information Technology and Computer Science for their cooperation and assistance throughout the development of this project. We would also like to acknowledge our friend Mr. Sudeep Kumar Mishra for his continuous guidance and support throughout this project.

Finally, we express our deepest gratitude to our parents, friends, and classmates for their constant motivation, understanding, and support during the completion of this project.

We sincerely acknowledge everyone who contributed, directly or indirectly, to the successful completion of **NetSentinel AI**. Their support and encouragement have made this project a valuable and rewarding academic experience.

INDEX

No.	Description	Page No.
1.	**About Department of M.Sc.IT**	1
2.	**Project Profile** 2.1 Project Definition 2.2 Project Description 2.3 Existing System / Work Environment 2.4 Problem Statements 2.5 Need for New System 2.6 Proposed System & Features 2.7 Scope 2.8 Outcomes 2.9 Tools & Technology used 2.10 Project Plan	8-13
3.	**Requirement Analysis** 3.1 Feasibility Study 3.2 Users of the System 3.3 Modules 3.4 Process Model 3.5 Hardware & Software Requirements 3.6 Use Cases 3.7 Use Case Diagram	14-20
4.	**Design** 4.1 Use Case Scenarios 4.2 Diagrams 4.2.1 UML / DFD 4.2.2 Entity Relationship Diagram 4.3 Data Dictionary	21-27
5.	**Implementation** 5.1 Form Layouts 5.2 Report Layouts 5.3 Coding Convention (Business Logic)	28-50
6.	**Testing**	51-55

	6.1 Test Strategy 6.2 Test Cases	
7.	**Future Enhancements**	**56-57**
8.	**Bibliography**	**58-59**
9.	**Appendix**	**60-67**

1. ABOUT DEPARTMENT OF M.Sc. IT

1.1 Parul University

Parul University is a multidisciplinary institution located in Vadodara, Gujarat, India. The university provides academic programmes across computing, engineering, management, health sciences, design, and other disciplines. The Faculty of IT & Computer Science supports learning in software engineering, information technology, data management, networking, and emerging areas of computing.

The academic environment encourages students to connect classroom concepts with practical implementation. Projects provide a structured opportunity to identify a problem, study existing approaches, translate requirements into a design, build a working solution, test expected behaviour, and communicate outcomes through formal documentation.

1.2 Programme Context

The Master of Science in Information Technology programme develops advanced understanding of information systems, programming, databases, networking, software design, and applied computing. Semester projects help students practise requirement analysis, system modelling, implementation planning, validation, and technical reporting. Cybersecurity projects are especially relevant because they require attention to confidentiality, integrity, availability, access control, logging, and responsible handling of data.

NetSentinel AI aligns with this learning context by combining a web application with data processing and a machine-learning classification workflow. It demonstrates how a security-oriented problem can be decomposed into user-facing functions, backend services, an inference pipeline, and persistent records.

1.3 Project Relevance

Intrusion detection is a core activity in network defence. Network administrators and security Users need to examine activity for patterns that may indicate scanning, denial-of-service behaviour, brute-force attempts, or other anomalous events. A dataset-based machine-learning application can support academic experimentation by making the classification workflow visible and repeatable. It does not replace network sensors, incident-response procedures, or expert analysis; rather, it provides a platform for exploring how labelled records can be processed and classified.

The project also provides practical exposure to secure file upload, validation of untrusted input, model loading, result presentation, database records, and the distinction between experimental predictions and operational security alerts. These are useful foundations for further study in security operations, penetration testing, threat detection, and applied machine learning.

2. PROJECT PROFILE

2.1 Project Definition

NetSentinel AI – AI-Based Intrusion Detection System is a web-based academic application designed to analyse network-traffic records from a structured CSV dataset and classify the records using a trained machine-learning model. It provides an authenticated interface for uploading data, preprocessing supported fields, generating predictions, and presenting a summary for review. The application uses Django for web application functionality, Python for core logic, Pandas and NumPy for data handling, scikit-learn for machine-learning workflows, SQLite for application persistence, and Matplotlib for selected visual summaries where implemented.

The current project scope is dataset-driven: a user supplies traffic records in a supported CSV format. Although live packet monitoring is a possible future enhancement, the report does not claim that the current implementation continuously captures traffic from a production network. The distinction is important because offline classification and live network intrusion detection have different operational requirements, latency constraints, data collection mechanisms, and validation methods.

2.2 Project Description

The application is organised around a simple workflow. A user first signs in through the web interface. After authentication, the dashboard provides access to the dataset-upload and prediction features. The uploaded file is validated before it is processed. The preprocessing stage prepares columns and values for the model, handling supported transformations and checking whether the required features are present. The prediction module loads the trained estimator and generates a classification for each eligible record. The application then displays a result summary and persists relevant metadata so the user can revisit previous predictions.

The project demonstrates the integration of three areas. The first is web engineering: authentication, forms, request handling, templates, and user feedback. The second is data engineering: CSV parsing, validation, feature alignment, and safe handling of malformed inputs. The third is machine learning: applying a trained classifier consistently to records that have been transformed into the representation expected by the model. Reliable integration requires that the preprocessing used during prediction match the preprocessing used when the model was trained.

NetSentinel AI is intended for educational demonstrations, controlled experiments, and preliminary analysis of labelled or appropriately prepared network datasets. Any use beyond a controlled academic setting requires additional validation, monitoring, access controls, dataset governance, and review by qualified security personnel.

2.3 Existing System / Work Environment

Existing intrusion-detection approaches include signature-based network intrusion detection systems, rule-based monitoring, anomaly detection, and machine-learning classifiers. Signature-based systems compare observed

events with known patterns and can be effective for threats covered by current signatures. Rule-based systems provide explicit logic that Users can inspect and maintain. Anomaly-based systems identify deviations from a baseline, although unusual legitimate behaviour can generate false positives. Supervised machine-learning models learn a mapping between input features and known labels from a labelled training set.

In an academic workflow, datasets are often analysed with standalone scripts or notebooks. While this is flexible, it can separate data preparation, prediction, and result review across different tools. Users may need to run scripts manually, remember command-line parameters, and locate output files. A web interface can make the sequence easier to demonstrate and can centralise prediction metadata. It also introduces its own responsibilities, including authentication, file-upload constraints, error handling, and database consistency.

NetSentinel AI brings the dataset-based workflow into a single application. It is not assumed to outperform established IDS products, and the project does not claim that a model trained on one dataset will generalise to every network. Its value is as a modular educational implementation that makes the pipeline accessible and provides a foundation for controlled evaluation.

2.4 Problem Statements

1. Network traffic datasets can contain many records and features, making manual inspection slow and inconsistent.
2. Rule-based detection may not capture every pattern represented in a labelled dataset, especially when the rule set is incomplete or outdated.
3. Running separate preprocessing and prediction scripts can make the analysis workflow difficult to reproduce for novice users.
4. Users need a clear interface to upload a dataset, receive feedback about invalid files, and review classification summaries.
5. Prediction outputs should be recorded with enough metadata to support traceability and comparison across experiments.
6. Model performance must be measured with appropriate metrics; a single accuracy value can hide poor detection of minority attack classes.
7. Input files and model outputs must be handled safely so malformed data does not cause application failures or compromise stored records.

2.5 Need for New System

A centralised dataset-analysis interface can reduce friction in academic experiments by bringing upload, preprocessing, prediction, and reporting into a repeatable sequence. It can help users understand how records move through a classification system and make results easier to review. The application also provides a practical setting for studying the security implications of file handling, user authentication, error messages, and persistent history.

The need is therefore not simply to add a machine-learning model to a website. It is to create a coherent workflow where input validation, feature consistency, model inference, and result presentation are connected. The application should communicate uncertainty and scope clearly: a model prediction is an estimate based on learned patterns, not proof that a host is compromised or that a traffic record is malicious in every operational context.

2.6 Proposed System & Features

The proposed system uses Django to manage web requests and application views. Authenticated users can access a dashboard, upload supported CSV files, request predictions, and review stored reports. The data-processing layer checks file structure and prepares model inputs. The inference layer applies a trained scikit-learn estimator and returns labels or supported class outputs. The persistence layer stores relevant application records in SQLite. Visual summaries can help communicate class distributions or prediction counts when the relevant values are available.

Feature	Description	Expected benefit
User authentication	Login and logout through the web application	Restricts access to registered users and protected functions
Dashboard	Central navigation and recent activity summary	Improves usability and makes core functions discoverable
Dataset upload	Accept supported CSV network-traffic files	Provides a repeatable input path for analysis
Input validation	Check extension, size, columns, and parseability	Reduces malformed input and processing failures
Preprocessing	Clean and align features with the trained model schema	Improves consistency between training and inference
ML prediction	Apply a trained estimator to eligible records	Produces predicted labels for review
History and reports	Persist prediction metadata and present summaries	Supports traceability across experiments
Visual analytics	Display appropriate counts or charts	Makes results easier to interpret

2.7 Scope

The scope includes a web-based application for dataset-driven intrusion prediction. It covers user authentication, CSV upload, input checks, preprocessing, model inference, display of prediction outcomes, database storage of application records, and report/history views to the extent implemented in the source application. The technology baseline identified in the project materials is Python, Django, scikit-learn, Pandas, NumPy, SQLite, Matplotlib, and Wireshark for supporting packet-analysis work.

The project does not automatically imply live packet capture, inline traffic blocking, a distributed sensor architecture, cloud-scale deployment, or a fully operational security information and event management (SIEM) integration. These features require additional engineering and are treated as future work unless verified in the actual source code and deployment.

2.7.1 Intended Users

1. Students and faculty members conducting controlled cybersecurity experiments.
2. Cybersecurity learners studying intrusion-detection pipelines and classification metrics.
3. Researchers preparing reproducible experiments on supported network datasets.
4. Users using the application as a supplementary offline dataset-inspection tool.

2.7.2 Assumptions and Constraints

The application assumes that the uploaded dataset contains the features required by the trained model or can be transformed into that schema. Prediction quality depends on training data quality, label correctness, class balance, feature definitions, and similarity between training and evaluation environments. SQLite is suitable for a small academic deployment, but high concurrency and large-scale operational use may require a different database and a more robust deployment architecture. Dataset licences and privacy obligations must be checked before use.

2.8 Outcomes

The expected outcome is an integrated demonstration of a web application that receives a network dataset and applies a machine-learning classifier to the supported records. The system should provide understandable feedback at each major stage and preserve sufficient information to review a prediction run. The academic outcome is a documented implementation that connects requirement analysis, system design, web engineering, data processing, machine learning, and software testing.

The quality of the outcome should be judged through functional tests, data-validation tests, and a transparent evaluation of the model. Model accuracy, precision, recall, F1-score, confusion matrix, and class-wise results should be taken from the actual training or evaluation run. They are intentionally not fabricated in this report. Where those values are not available, the report provides a measurement framework and indicates the values that must be inserted after execution.

2.9 Tools & Technology Used

Tool / technology	Purpose	Justification / use in project
Python	Core programming language	Provides a broad ecosystem for data handling, ML, and security scripting.

Django	Web application framework	Provides request routing, views, templates, ORM, and built-in authentication support.
scikit-learn	Machine-learning toolkit	Supports training, preprocessing pipelines, classification, and evaluation metrics.
Pandas	Tabular data handling	Reads CSV data, inspects columns, handles missing values, and prepares DataFrames.
NumPy	Numerical operations	Supports efficient numeric arrays and data transformations.
SQLite	Relational database	Provides lightweight local persistence suitable for a small academic deployment.
Matplotlib	Data visualisation	Can render plots for prediction summaries and model evaluation.
Wireshark	Packet-analysis utility	Supports controlled inspection of packet captures and helps explain traffic features.
HTML/CSS/templates	Web presentation	Renders forms, dashboard views, and result pages.
Git/GitHub	Version control	Tracks code changes and supports collaborative development.

2.10 Project Plan

The project plan follows an incremental software-development process. Initial work defines the problem and reviews relevant IDS concepts. Requirement analysis identifies users, functions, constraints, and evaluation needs. System design then describes the application layers, workflows, data entities, and model pipeline. Implementation integrates the web interface, preprocessing logic, classifier, and persistence. Testing verifies the functional flow and checks the model using suitable metrics. Documentation records the decisions, limitations, and opportunities for future enhancement.

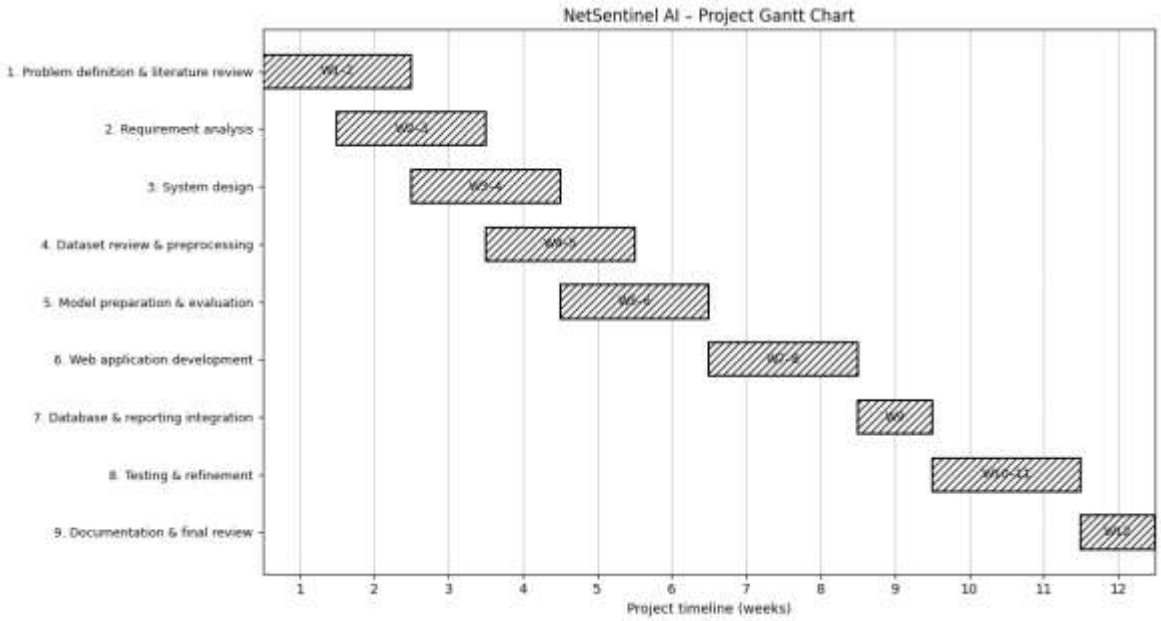


Fig 2.10: Project Timeline Gantt Chart

3. REQUIREMENT ANALYSIS

Requirement analysis identifies what the system must do, how well it must operate, and which constraints affect its design. For NetSentinel AI, the analysis covers the user-facing workflow, dataset constraints, model integration, result persistence, and safe handling of uploaded files. Requirements in this chapter are intended to guide verification; they should be checked against the implemented source code before the project is represented as production-ready.

3.1 Feasibility Study

Technical Feasibility

The project is technically feasible for a small academic deployment because its core technologies are available as mature open-source tools. Django provides the web framework, scikit-learn provides common supervised-learning algorithms and metrics, Pandas and NumPy support tabular processing, and SQLite provides local persistence. The major technical risk is not tool availability but keeping the feature schema, preprocessing operations, and model version consistent from training through inference.

Operational Feasibility

The web-based workflow is suitable for students and users familiar with basic browser interaction. A dashboard, upload form, and results page reduce the need to run separate scripts manually. Operational feasibility depends on clear instructions, informative validation errors, reasonable file-size limits, and the ability to recover from invalid data without losing existing history. Users should be told that the tool performs dataset-based classification and does not independently confirm a live compromise.

Economic Feasibility

The technology stack is largely open source and can be developed on a standard laptop or desktop. This reduces licensing cost for the academic prototype. Costs may still arise from hardware, storage, maintenance, time spent preparing datasets, and future hosting. Any dataset acquisition must comply with its licence and may introduce separate costs or restrictions.

Schedule Feasibility

A twelve-week plan is plausible for a bounded academic prototype if dataset preparation, model integration, and interface work are prioritised early. The schedule becomes less predictable if the project expands to live packet capture, streaming analysis, model explainability, or deployment in a real network. The schedule should therefore treat future features as separate milestones rather than mandatory elements of the current deliverable.

3.2 Users of the System

NetSentinel AI defines two primary application roles: User (user) and Administrator (Admin). Each role has specific responsibilities and access permissions based on the functions provided by the application.

1. User (user) Role

The user role is designed for standard users or security Users who use NetSentinel AI to analyze network traffic, detect potential intrusions, and review prediction results.

Responsibilities

1. Authentication: Register an account, sign in using valid credentials, and sign out securely.
2. Dataset Upload: Upload supported CSV files containing network traffic records for batch intrusion analysis.
3. Live Traffic Monitoring: Discover available network interfaces and initiate live traffic capture when the required capture subsystem and permissions are available.
4. Traffic Simulation: Generate simulated network traffic for testing and evaluate it using the machine-learning model.
5. Intrusion Prediction: Submit network traffic data for classification and obtain prediction results.
6. Result Analysis: Review prediction summaries, detected attack categories, confidence values, and available feature-importance information.
7. Alert Management: View security alerts associated with permitted records and resolve alerts when authorized.
8. Traffic Log Review: Examine accessible traffic logs to understand network activity and detected threats.
9. Prediction History: Review previous dataset-analysis runs and access their corresponding results.
10. Reports and Downloads: View dashboards, charts, and available reports, and download scored CSV results.
11. Personal Settings: Manage available profile settings, interface themes, and other user-level preferences.

Access Expectations

The user can access standard application features and permitted records associated with their account. Where ownership-based access control is implemented, users should only be able to view or manage their own traffic logs, alerts, and prediction records. They must not be allowed to manage other users, change application roles, initiate administrator-only model retraining, or execute restricted administrative testing operations.

2. Administrator (Admin) Role

The Admin role is designed for authorized personnel responsible for application administration, security monitoring, user management, and machine-learning model operations.

Responsibilities

1. Administrative Authentication: Sign in using an authorized administrator account and access privileged functions.
2. User Management: View user accounts, create or update accounts, assign roles, and manage user access through the administrative interface.
3. Global Security Monitoring: Monitor system-wide traffic logs, security alerts, prediction activity, and aggregated dashboard metrics.
4. Alert Supervision: Review security alerts across authorized records and resolve alerts when required.
5. Operational Reporting: Access system-wide reports, attack-distribution statistics, and alert-resolution indicators.
6. Model Performance Evaluation: Review model evaluation metrics, including accuracy, precision, recall, and F1-score.
7. Model Retraining: Initiate the supported model-retraining process and examine its status and evaluation results.
8. Automated System Testing: Execute the available regression test suite to verify application functionality and review the resulting test reports.
9. Application Configuration: Access administrative settings and operational controls provided by the application.
10. Audit and Oversight: Review authorized application records to support security monitoring, troubleshooting, and system evaluation.

Access Expectations

The administrator has privileged access to administrative functions and authorized system-wide records. In addition to administrative operations, the administrator may use standard user features where permitted. Access to user management, model retraining, global monitoring, and automated testing must be enforced through server-side authorization checks.

3. Role-Based Access Control

NetSentinel AI should enforce role-based access control to ensure that users and administrators can perform only the operations permitted to their roles. Standard users should have access to their own permitted workflows and records, while administrators should have access to additional management and monitoring capabilities.

In the Django application, these permissions should be enforced through authentication checks, role-validation middleware, appropriate view-level authorization, and ownership filtering for user-specific records. Merely hiding administrative buttons from standard users is insufficient to protect privileged operations.

3.3 Modules of the System

1. Authentication Module

The authentication module accepts credentials and uses the web framework's authentication mechanisms to verify them. It should reject invalid credentials without exposing sensitive details, protect authenticated views, and support logout. Passwords should be stored using the framework's password-hashing facilities rather than plaintext. Session cookies and deployment settings should be configured appropriately for the environment.

2. Dashboard Module

The dashboard acts as the main navigation point after sign-in. It can show links to upload, prediction, reports, and history, together with recent activity where that data is available. Dashboard values should be derived from persisted records or the current session and should not display fabricated statistics. If the application does not yet implement a live-monitoring feed, the dashboard must not imply that live traffic is being monitored.

3. Dataset Upload Module

The upload module accepts a supported CSV file. Validation should check that the upload exists, the extension and content are appropriate, the file is within configured size limits, and the file can be parsed safely. A filename extension alone is not proof of content type. Uploaded data should be stored in a controlled location or processed in memory according to the application design, and temporary files should be removed when no longer needed.

4. Data Preprocessing Module

Preprocessing aligns raw input with the model's expected representation. Operations may include selecting required columns, converting data types, handling missing values, encoding categorical fields, normalising numeric fields where the trained pipeline requires it, and preserving the expected feature order. The same transformation logic used at training time must be reused at inference time. Silent replacement of unknown or malformed values should be avoided because it can distort predictions.

5. Machine Learning Prediction Module

The prediction module loads a trained model or pipeline and applies it to eligible records. It should verify that the artefact is present and compatible, handle prediction errors, and return the output in a format that the application can display and store. If the model produces only binary labels, the interface should not present unsupported attack subtypes or confidence scores. A prediction is a model output and should be interpreted alongside dataset context and evaluation evidence.

6. Database Management Module

SQLite is used for application persistence in the documented stack. Records may include users managed by Django, dataset metadata, prediction summaries, timestamps, and report history. The exact fields depend on the

implemented schema. Database operations should use the framework ORM or parameterised queries, and the application should handle database errors without exposing internal stack traces to end users.

7. Reporting Module

The reporting module organises prediction results for review. A report can include the uploaded dataset name, run timestamp, total records processed, counts by predicted class, and references to stored output. Metrics such as precision or recall belong to model evaluation and should not be confused with per-file prediction counts. A report should identify the dataset and model version where those values are available.

3.4 Process Model

An incremental SDLC model is appropriate for this project. Requirements and architecture are defined first, after which data processing, model integration, web views, and reporting can be implemented in small increments. Each increment is tested before it is integrated into the complete workflow. This approach allows problems in the data schema or model interface to be identified before the final demonstration.

SDLC stage	NetSentinel AI activity	Verification evidence
Requirement analysis	Define upload, prediction, history, security, and usability needs	Requirement checklist and use cases
Design	Model layers, data flow, entities, interfaces, and error paths	Architecture, DFD, ERD, and activity diagrams
Implementation	Develop Django functions and connect preprocessing and inference	Code review and module-level tests
Integration	Connect upload, prediction, persistence, and reporting	End-to-end test runs
Testing	Test normal, invalid, boundary, and failure conditions	Test log and captured outputs
Maintenance	Correct defects and update datasets/models when required	Version history and change notes

3.5 Hardware & Software Requirements

Hardware Requirements

Component	Minimum practical baseline	Recommended for development
Processor	Modern dual-core CPU	Quad-core or better

Memory	8 GB RAM	16 GB RAM or more for larger datasets
Storage	20 GB free storage	SSD with additional room for datasets and model artefacts
Display	1366 × 768 resolution	Full HD display
Network	Local network or internet for setup	Stable internet for package and documentation access

Software Requirements

Software	Role	Notes
Supported Python version	Runtime	Use the version compatible with project dependencies
Django	Web framework	Use supported security updates
scikit-learn	Model inference and evaluation	Model artefacts should be version-compatible
Pandas / NumPy	Data processing	Pin versions for reproducibility
SQLite	Application database	Suitable for small local deployments
Matplotlib	Charts	Use only where report visualisation is implemented
Web browser	Client interface	Current browser with JavaScript/CSS support
Git	Version control	Exclude secrets and sensitive datasets from commits

3.6 Use Cases

Use Case	**Primary Actor**	**Precondition**	**Expected Outcome**
View Landing Page	Anonymous User	Application is accessible.	Public landing page is displayed.
Register Account	Anonymous User	Registration page is available and details are provided.	Account is created with the default user role.

Sign In / Sign Out	User / Administrator	Valid account exists for sign-in; active session exists for sign-out.	Session is established or terminated; errors are displayed when appropriate.
View Workspace Dashboard	User	User is authenticated.	Personal dashboard displays owned alerts, logs, prediction runs, and telemetry.
View Global SOC Dashboard	Administrator	Administrator is authenticated.	Global metrics and user activity are displayed.
View Telemetry Charts	User / Administrator	Authenticated session exists.	User sees own telemetry; administrator sees global trends, protocol shares, and severity distribution.
Discover Interfaces and Monitor Status	User / Administrator	Authenticated session exists.	Available Wireshark/TShark interfaces and monitoring status are displayed.
Capture Live Network Traffic	User / Administrator	Authenticated session exists and capture is available.	Packets are captured and network features are extracted for prediction.
Simulate Network Traffic	User / Administrator	Authenticated session exists.	Synthetic network flows are generated and scored by the ML pipeline.
Upload CSV Dataset	User / Administrator	Authenticated session exists and a CSV dataset is available.	Dataset is validated and multiple network-flow records are scored.
Inspect Prediction Results	User / Administrator	A prediction run exists.	Flow summary, dominant classification, confidence, and feature importance are displayed; access is ownership-filtered for Users.

Download Scored CSV	User / Administrator	A prediction run exists and is accessible to the actor.	Enriched CSV containing scored predictions is downloaded.
Review Dataset Run History	User / Administrator	Stored prediction runs exist.	Historical batch prediction records are listed according to role permissions.
View Security Alerts	User / Administrator	Security alerts are available.	Alerts are listed by severity; Users see their own alerts, while Administrators see all alerts.
Resolve Security Alert	User / Administrator	A target alert exists and is accessible to the actor.	Alert is marked as resolved; Users can resolve their own alerts, while Administrators can resolve any alert.
Explore Traffic Logs	User / Administrator	Traffic log records exist.	Packet and flow details, predictions, and confidence values are displayed according to access permissions.
Review Threat Analytics Reports	User / Administrator	Relevant alert and traffic data are available.	Attack distribution and alert-resolution metrics are displayed for the permitted data scope.
Manage Workspace Settings	User / Administrator	Authenticated session exists.	Account and theme preferences are managed; administrator options are shown according to role.
View Model Metrics	User / Administrator	Model evaluation metrics are available.	Accuracy, precision, recall, and F1-score are displayed.
Retrain Random Forest Model	Administrator	Administrator is authenticated and retraining is available.	Background retraining runs and updated model artefacts and metrics are saved.

Manage User Roles and Permissions	Administrator	Administrator is authenticated.	User accounts and roles are managed with safeguards against unsafe role changes.
Execute Automated Test Suite	Administrator	Administrator is authenticated and the test suite is available.	Regression tests TC-01 to TC-10 are executed and results are recorded.
Download Test Run Report	Administrator	A test-run report is available.	The test report is exported as a downloadable Markdown file.

4. DESIGN

System design converts the requirements into a set of cooperating components and defines the structure of data movement through the application. The design separates the presentation layer, Django request handling, dataset-processing logic, model inference, and persistence. This separation is important because the model pipeline may evolve independently of the user interface, while database and security responsibilities remain explicit.

4.1 Use Case Scenarios

4.1.1 Authentication Scenario

The user opens the login page and enters credentials. The application validates the request and delegates credential verification to the authentication framework. If the credentials are valid, the session is established and the user is redirected to the dashboard. If validation fails, the application returns a controlled message and does not reveal whether a specific account exists. Protected views must verify authentication independently rather than relying on the interface to hide links.

4.1.2 Dataset Upload Scenario

The authenticated user opens the upload page, selects a CSV file, and submits the form. The server checks the file size and supported format, attempts to parse the contents, and verifies the presence of required columns. If the file is invalid, the request ends with a clear error and no prediction is recorded. If it passes validation, the system prepares the dataset for preprocessing and prediction.

4.1.3 Prediction Scenario

The application receives a validated dataset and applies the configured preprocessing pipeline. It checks feature alignment and handles missing or unsupported values according to the training contract. The trained estimator generates predictions for the eligible records. The system aggregates supported counts, records metadata, and returns a result page. If the model artefact cannot be loaded or inference fails, the application should report a controlled failure rather than displaying stale or fabricated results.

4.1.4 Report and History Scenario

After a prediction run, the user opens the result or history view. The application retrieves the corresponding stored record and displays the dataset name, run time, and available result summary. The report should not silently combine metrics from unrelated experiments. If the detailed per-record output is not persisted, the report must not imply that it can be reconstructed from a summary-only record.

4.2 Diagrams

4.2.1 System Architecture Diagram

The architecture separates the browser-facing presentation layer from Django application logic. Upload handling and validation feed into preprocessing, which prepares the feature matrix for the trained machine-learning pipeline. The results layer returns predictions to the user and stores relevant metadata through the persistence layer. This logical design does not by itself imply distributed deployment or live packet capture.

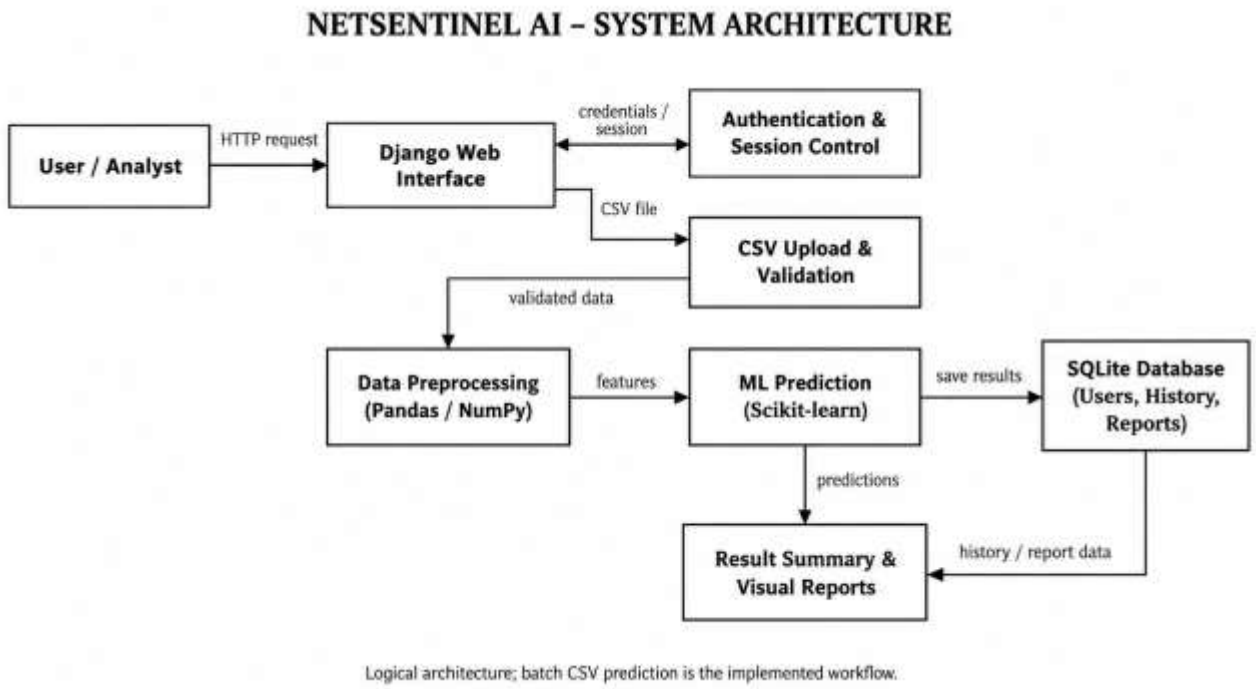


Figure 4.2.1 – System Architecture Diagram

4.2.2 Use Case Diagram

The use-case view identifies the major actions and functionalities supported by NetSentinel AI from the perspective of its users and administrators. It focuses on the system's functional requirements and illustrates how each actor interacts with the application. The diagram is deliberately kept at the functional level to provide a clear overview of the system's capabilities without displaying every internal class, database operation, or implementation detail. Administrative functions, such as user management, model retraining, and automated testing, should be included only when the corresponding permissions and application interfaces are implemented. This approach ensures that the use-case diagram accurately represents the project's functionality, role-based access restrictions, and intended system behavior.

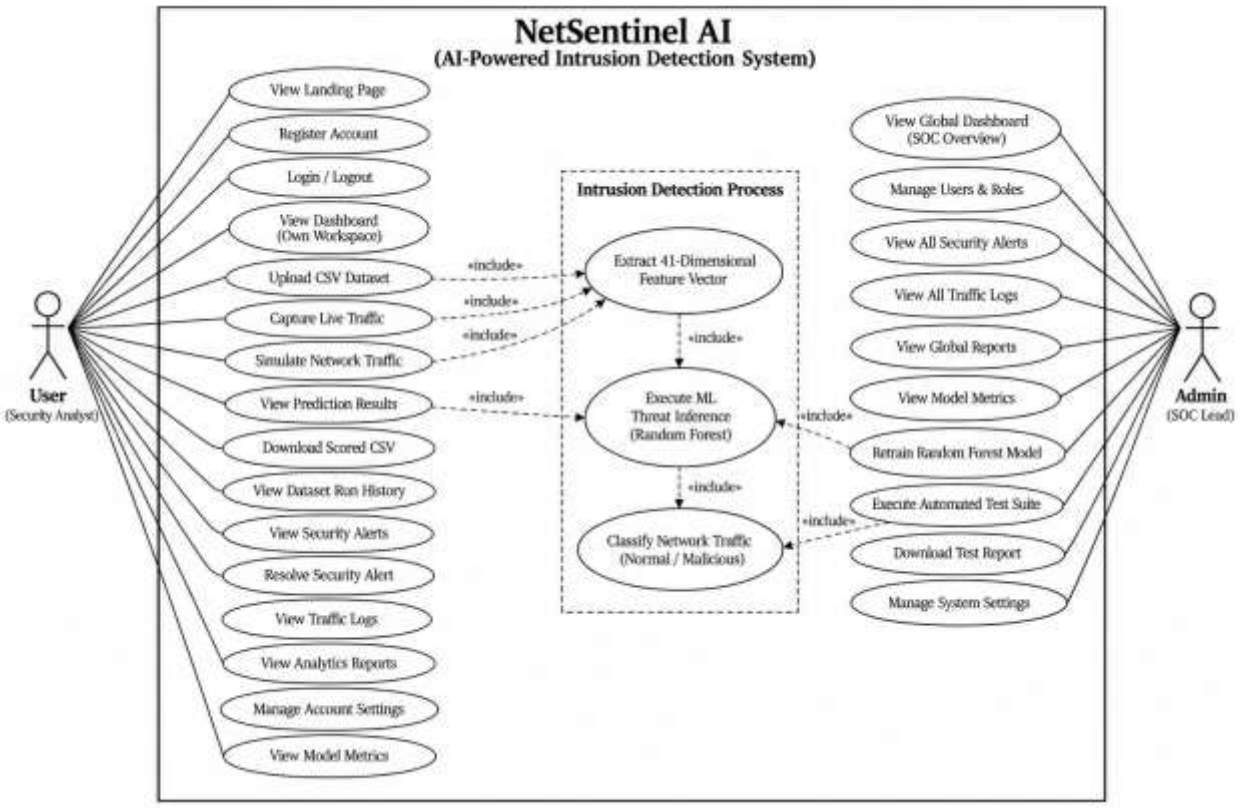


Figure 4.2.2 – User Interaction Use Case Diagram

4.2.3 Data Flow Diagram

The Level 0 DFD treats NetSentinel AI as a single process. The user supplies credentials and a dataset, and the system returns status, predictions, and report summaries. The diagram focuses on data entering and leaving the system boundary.

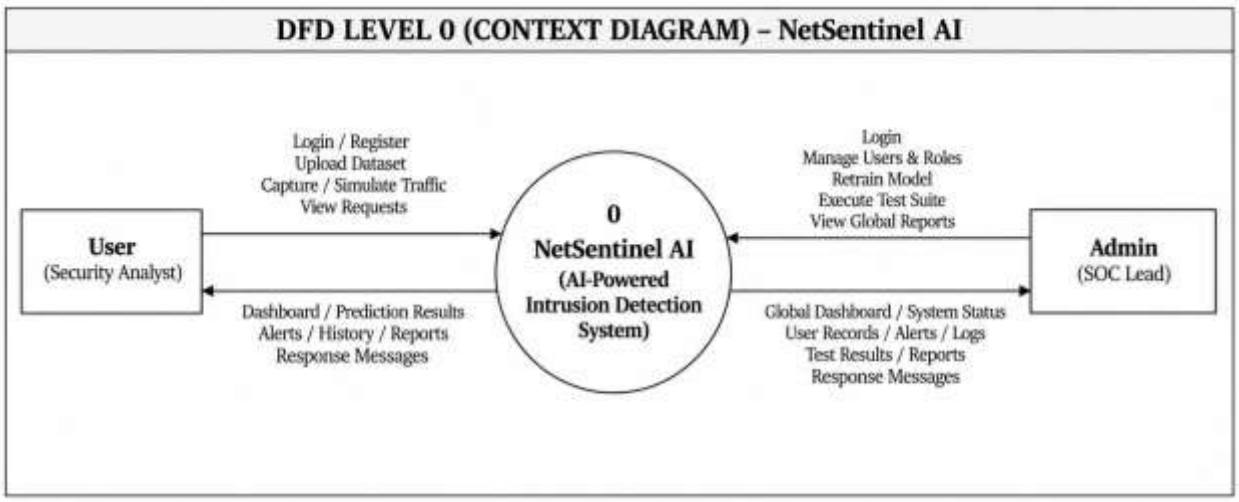


Figure 4.2.3 (a): DFD Level 0

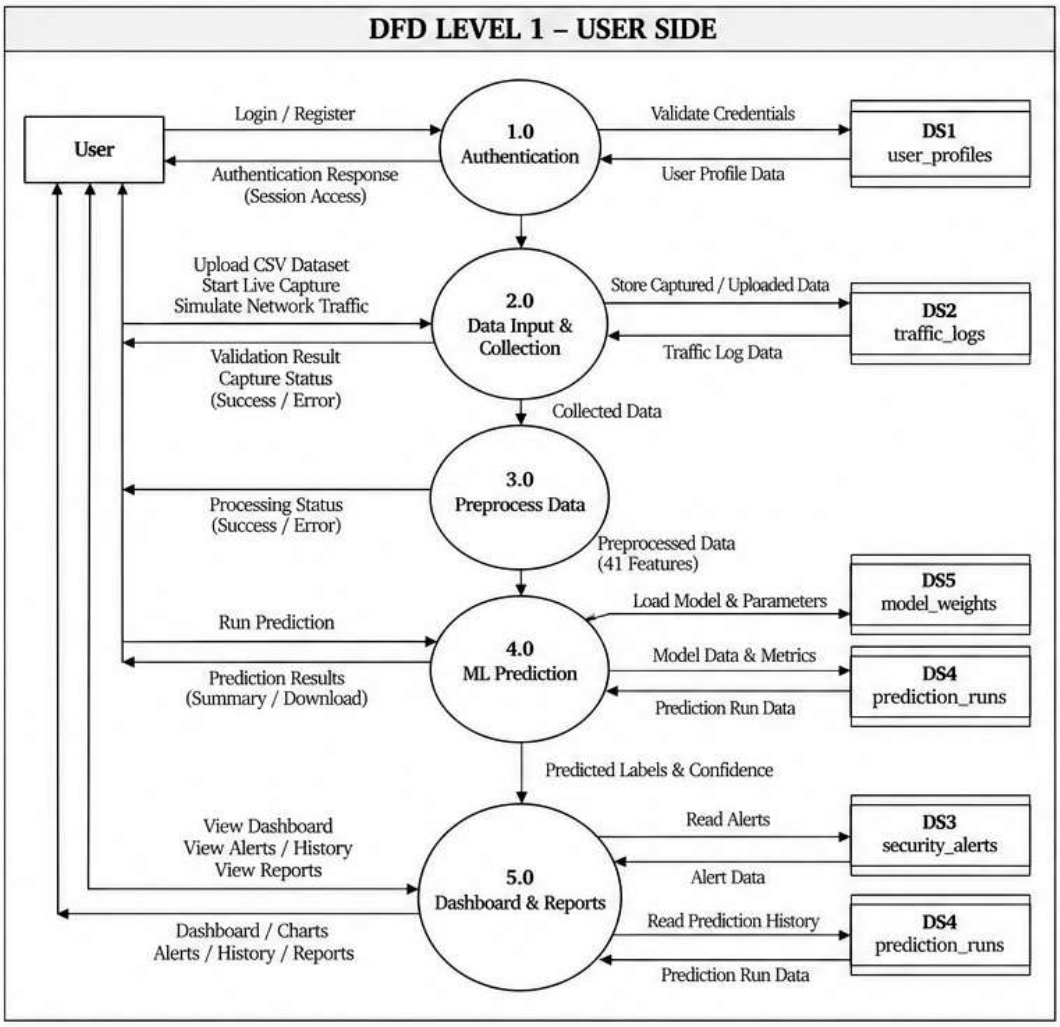


Figure 4.2.3 (c): DFD Level 1 – User Side

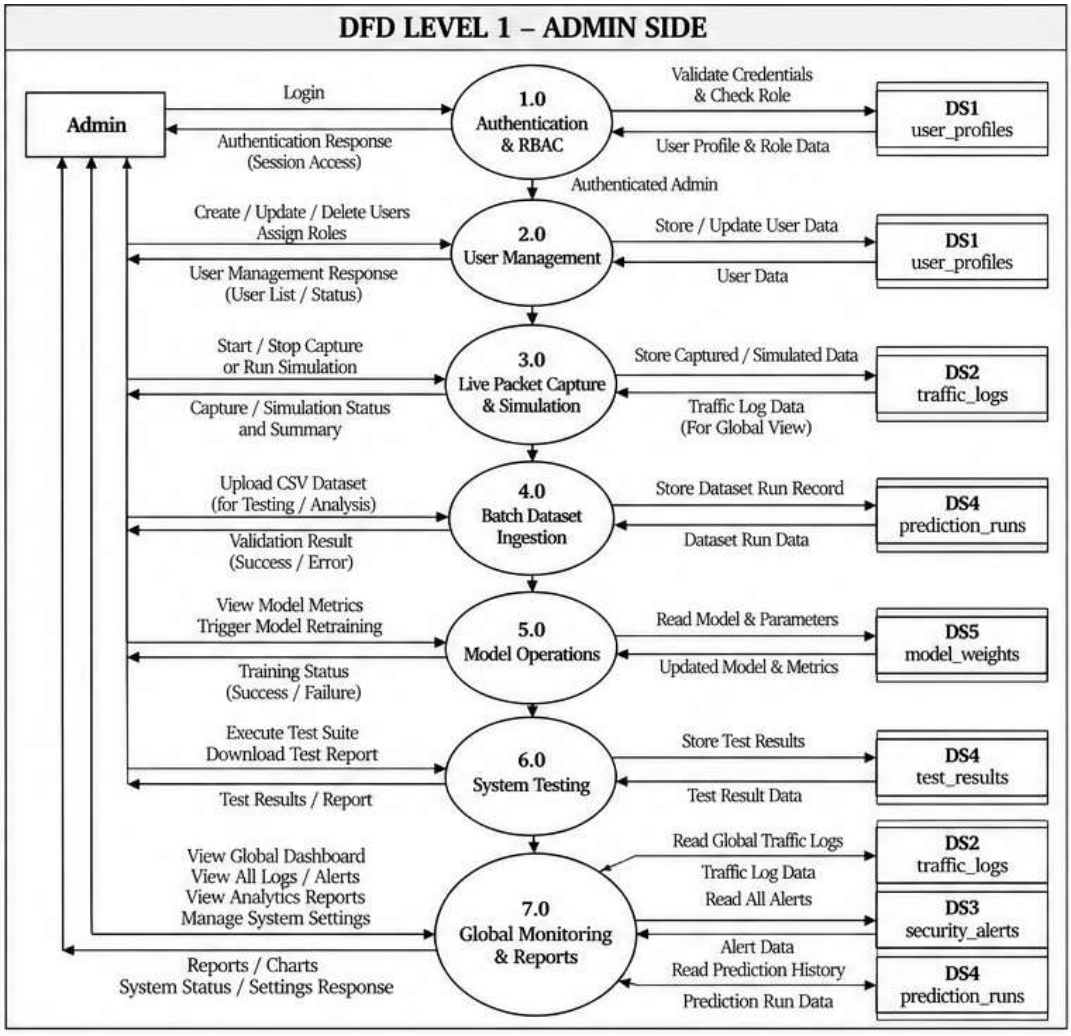


Figure 4.2.3 (d): DFD Level 1 – Admin Side

The Level 1 DFD decomposes the system into authentication, upload validation, preprocessing, prediction, persistence, and reporting processes. The data stores represent the logical records needed by the workflow. The DFD should be interpreted as a functional data-flow model rather than a physical network topology.

4.2.4 Machine Learning Flow Diagram

The machine-learning flow starts with labelled training data, applies the training-time preprocessing, splits data into training and evaluation partitions, fits the selected estimator, and evaluates it with class-sensitive metrics. At inference time, the same preprocessing contract is applied to the uploaded records before the trained estimator generates labels. Training and inference should be kept separate so a user upload cannot accidentally retrain or alter the deployed model.

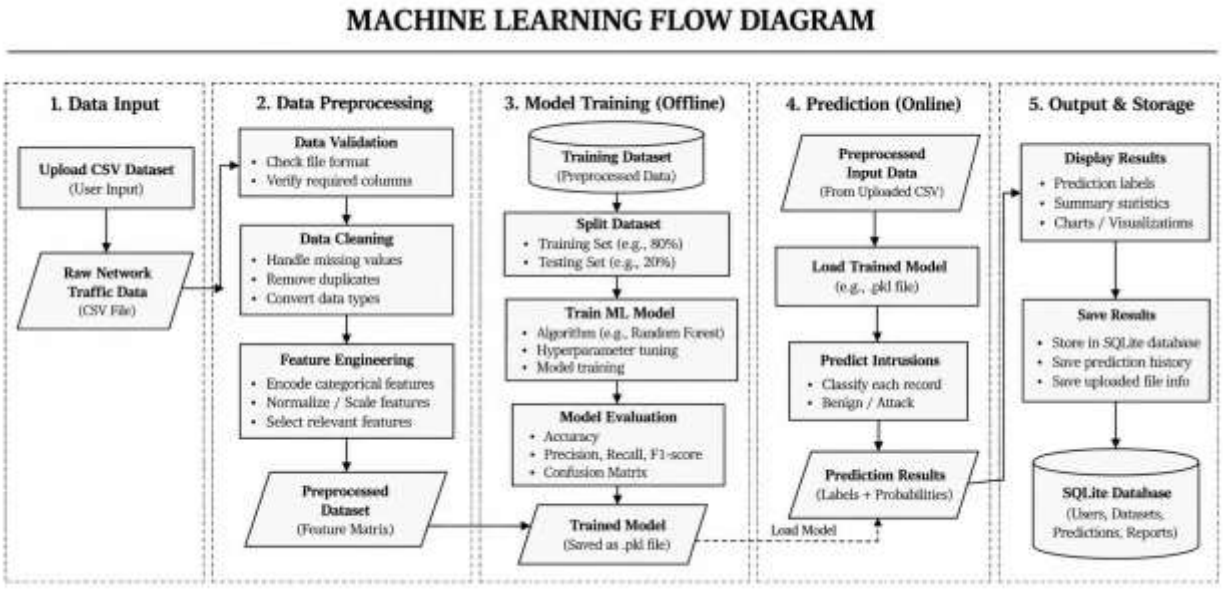


Figure 4.2.4 – Machine Learning Training and Prediction Flow

4.3 Entity Relationship Diagram

The ERD presents the persistent information needed to support authentication and prediction history. A user may be associated with multiple prediction runs. A prediction run may reference a dataset name, processing status, timestamp, summary values, and model metadata where these fields are implemented. Report records may be related to a prediction run if reports are stored separately. The actual Django models and migrations should be used to verify all attributes and cardinalities before final submission.

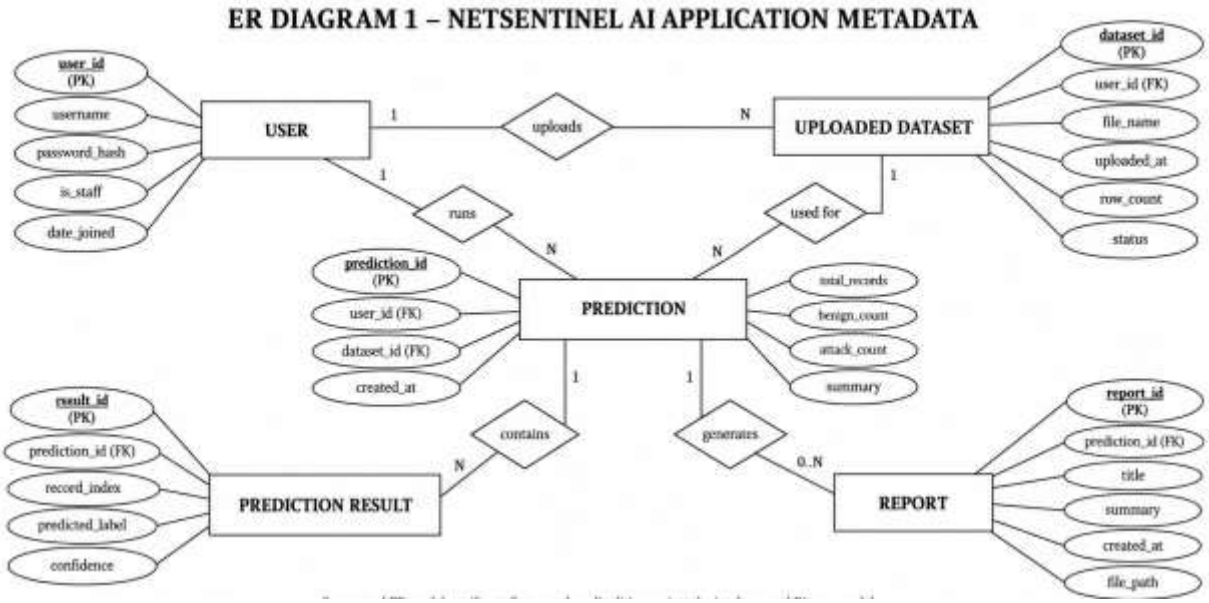


Figure 4.3 – Entity Relationship Diagram

4.4 Data Dictionary

Entity / field	Illustrative type	Purpose / notes
User	Django user model	Stores authentication identity and account metadata managed by the framework.
Dataset / upload metadata	String, size, timestamp, status	Records file identity and processing context where persisted.
Prediction run ID	Integer / UUID	Uniquely identifies a prediction execution.
Dataset name	String	Human-readable file name; should be treated as untrusted input.
Created timestamp	DateTime	Records when a prediction run was created.
Processing status	Choice / string	Indicates pending, successful, or failed processing if implemented.
Predicted class summary	JSON / text / related records	Stores counts or labels supported by the current schema.
Model version	String / metadata	Identifies the model artefact where versioning is implemented.
Report reference	Foreign key / path / text	Connects a report view or stored output to its prediction run, if implemented.

This dictionary is a design-level description based on the documented workflow. It must not be interpreted as a claim that every illustrative field already exists in the current database. Before submission, compare it with the Django models, migrations, and SQLite schema and remove or correct any field that is not implemented.

5. IMPLEMENTATION, FORM LAYOUTS & REPORT LAYOUTS

5.1 System Implementation

Implementation integrates the browser interface, Django application logic, data-preparation routines, trained estimator, and database operations. A modular implementation reduces the risk that request-handling code becomes tightly coupled to model logic. The web layer should validate the request and provide feedback, while the data-processing layer should prepare features and the inference layer should return predictions in a documented format.

Frontend Implementation

The frontend consists of HTML templates, CSS, forms, navigation, and result views rendered by the web application. The interface should make the main workflow obvious: sign in, select a dataset, start analysis, review the result, and open history. Form labels and validation messages should be accessible and understandable. Long-running processing should provide a clear status, and a failed upload should not appear as a successful prediction.

Backend Implementation

Django handles URL routing, view logic, authentication, request validation, template rendering, and persistence through its ORM. Views should remain small enough to test and maintain. Data processing and model inference can be placed in dedicated functions or modules so that they can be tested independently from HTTP requests. Unexpected exceptions should be logged securely and converted into controlled user-facing errors.

Database Implementation

SQLite supports local persistence for user-related data and prediction history. The Django ORM should be used for standard create, read, update, and delete operations. Database migrations should be tracked with the project, and the application should be tested against a clean database setup. Backups should be taken before schema changes where user data must be retained. For a high-concurrency production deployment, database choice should be reassessed.

Machine Learning Implementation

The model pipeline consumes a two-dimensional feature table whose columns and types must match the training contract. The selected scikit-learn estimator and any fitted preprocessing steps should be saved and loaded as a consistent artefact. The application should fail clearly if the artefact is missing or incompatible. Model files should come from a trusted build process; loading untrusted serialized model artefacts can execute code and must be avoided.

5.2 System Architecture During Implementation

At runtime, the user submits a request through the browser. Django verifies the session and validates the upload. The processing layer converts the file into a DataFrame and checks the required schema. The model pipeline generates predicted labels, and the reporting layer calculates supported summaries. Metadata is saved through the ORM, and the response is rendered as a result page. This is the logical request path; actual deployment details depend on the environment in which the project is run.

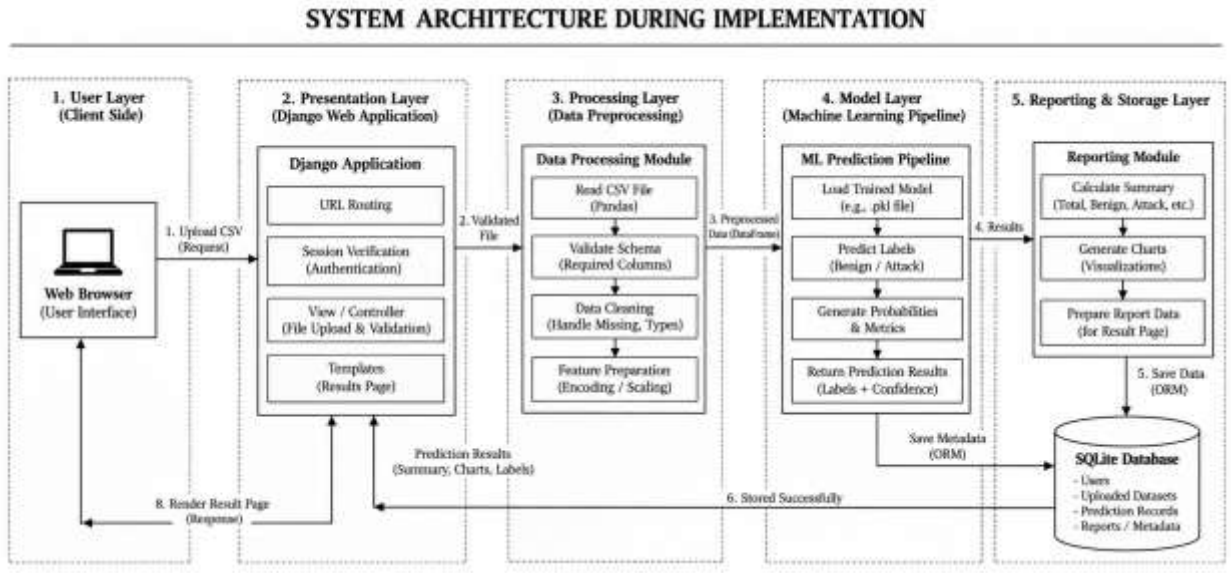


Figure 5.2 – Runtime Architecture of NetSentinel AI

5.3 Coding Convention (Business Logic)

The business logic follows a sequential processing contract. Authentication must precede protected actions. File validation must precede parsing and model inference. Preprocessing must produce the same feature order and representation used during model training. Prediction outputs must be checked before aggregation or persistence. Reporting must use the output from the current run and must not display cached values as though they belonged to a new upload.

1. Use descriptive function and variable names that identify the purpose of each operation.
2. Keep model loading, preprocessing, prediction, persistence, and rendering responsibilities separated.
3. Validate all user-controlled input on the server; browser-side checks are not sufficient.
4. Use Django authentication and ORM facilities instead of implementing custom password storage or assembling unsafe SQL.
5. Keep credentials, secret keys, and sensitive paths out of source control.
6. Log operational events without recording passwords, session tokens, or unnecessary packet payloads.

7. Pin dependency versions and document the Python and model versions used for evaluation.
8. Use deterministic preprocessing and a reproducible evaluation split when comparing model changes.

5.4 Algorithms and Techniques

5.4.1 Dataset Validation

Dataset validation checks that the uploaded object exists, is within the allowed size, can be parsed as a CSV file, and includes the required feature columns. Additional checks should identify duplicate or unexpected columns, invalid numeric values, unsupported labels, and empty files. Validation should be explicit: a file should not be silently accepted merely because Pandas can parse it.

5.4.2 Data Preprocessing

Preprocessing converts raw input into a representation accepted by the trained model. Depending on the dataset, it may include missing-value handling, type conversion, categorical encoding, numeric scaling, and feature selection. Preprocessing parameters such as means, scales, encoders, and column order should be learned from the training set and retained in the pipeline. Fitting a new scaler on each uploaded file can create inconsistent predictions and is generally inappropriate for a fixed trained model.

5.4.3 Supervised Classification

A supervised classifier learns from labelled examples. Let each training record be represented by a feature vector x and a target label y. The learning algorithm estimates a function f(x) that maps features to labels. At inference time, the trained function is applied to a compatible feature vector. The validity of the prediction depends on whether the input distribution resembles the training distribution and whether the labels and features represent the intended detection problem.

5.4.4 Evaluation Metrics

Accuracy measures the fraction of all predictions that are correct. Precision measures the proportion of predicted positives that are true positives. Recall measures the proportion of actual positives that are detected. F1-score is the harmonic mean of precision and recall. In intrusion detection, class imbalance can make accuracy misleading; therefore, class-wise precision, recall, F1-score, a confusion matrix, and an explicit definition of the positive class should accompany aggregate results.

Metric	Formula / interpretation	Why it matters
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall correctness; may hide minority-class failures.
Precision	TP / (TP + FP)	Indicates how many predicted attacks are correct.

Recall / detection rate	TP / (TP + FN)	Indicates how many actual attacks are detected.
F1-score	2 × Precision × Recall / (Precision + Recall)	Balances precision and recall.
False-positive rate	FP / (FP + TN)	Measures benign records incorrectly flagged as positive.
Confusion matrix	Counts TP, TN, FP, and FN	Shows the kinds of classification errors.

5.5 Form Layouts

5.5.1 Login Form

The login form accepts the username and password and submits them to the server-side authentication handler. The page should display clear but non-revealing error messages, preserve usability on common screen sizes, and avoid embedding credentials in URLs or logs. Login and logout routes must use the framework's session and CSRF protections as applicable.

Field / control	Purpose	Validation
Username	Identifies the account	Required; validated by authentication backend
Password	Verifies account access	Required; never echoed back or logged
Login button	Submits credentials	Disabled or protected against accidental duplicate submission where appropriate
Error message	Explains failed authentication	Must not expose sensitive account details

5.5.2 Dataset Upload Form

The upload form enables the user to select a CSV dataset and submit it for validation. It should clearly state the supported input format and any size or schema constraints. The server must independently verify the uploaded file, regardless of any browser-side restrictions. The page should provide a clear status for successful validation and specific corrective guidance when the file is rejected.

Field / control	Purpose	Validation / behaviour
File selector	Chooses the CSV file	Required; extension, content, size, and schema checked server-side
Upload / Analyse	Submits file for processing	Rejects empty or invalid submissions

Reset / Cancel	Clears the current selection	Does not delete stored history unless explicitly requested
Status message	Communicates result	Displays accepted, rejected, or processing state

5.5.3 Prediction Result Page

The result page should show the outcome of the current prediction run and distinguish it from model evaluation metrics. Suitable fields include the dataset name, timestamp, number of records processed, predicted class counts, and a link to the report or history record. Any probability or confidence value should be shown only if the selected model actually returns it and the value is correctly interpreted.

5.6 Report Layouts

A report should support traceability without overclaiming what the system knows. The dataset filename, run timestamp, processing status, model identifier, record count, and supported class summary provide useful context. If the application persists per-record predictions, the report may link to them; if it stores only aggregated data, it should clearly present the report as a summary. Evaluation metrics should be associated with a defined test dataset and evaluation run, not copied into each upload report without justification.

Report element	Content	Reason
Report identifier	Unique run or report reference	Enables traceability
Dataset information	Name and processed record count	Identifies the input context
Run timestamp	Date and time	Supports ordering and audit
Prediction summary	Counts by output label	Summarises model output
Model metadata	Model version where recorded	Supports reproducibility
Processing status	Success or controlled failure	Distinguishes complete runs from failed runs
Notes / limitations	Scope and interpretation	Prevents predictions being treated as definitive incident findings

5.7 Project Screenshots

The following interface screenshots and screen specifications document the operational visual layout of the NetSentinel AI platform. The application incorporates a modern Security Operations Center (SOC) visual language with dual-theme capability (light and dark mode), role-based view specialization, and real-time telemetry visualizations powered by Chart.js and Django.

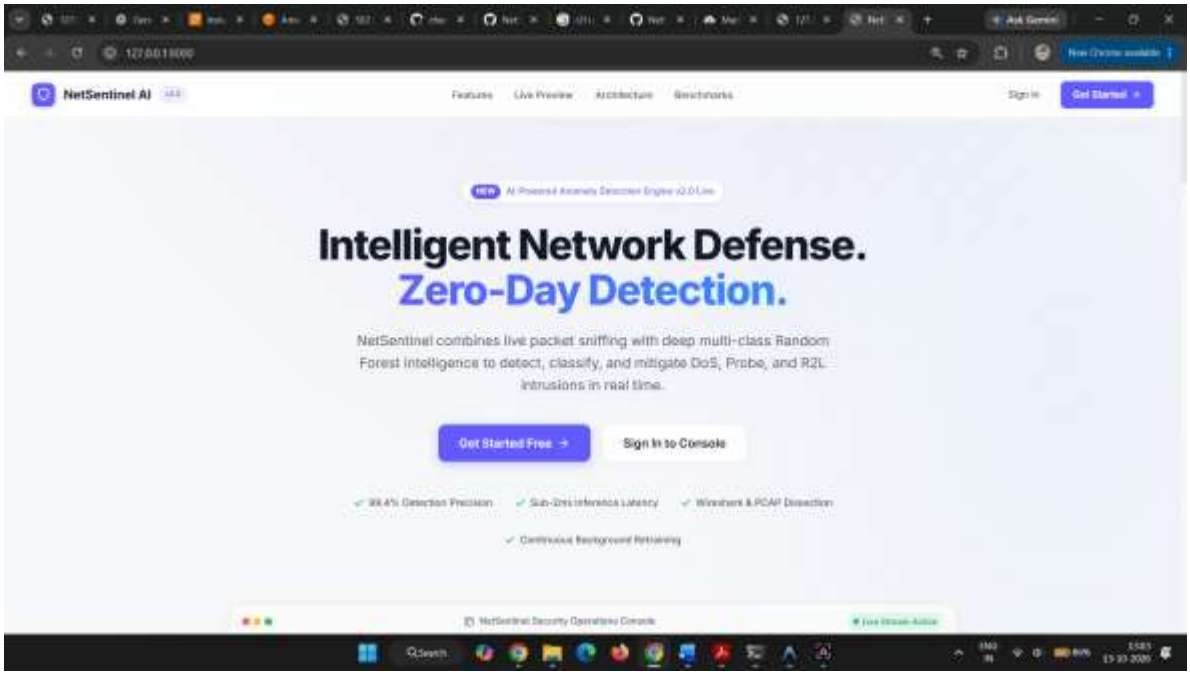
The interfaces are organized across three functional domains:

1. Public & Authentication Interfaces (accessible to all visitors)

2. Security User / User Role Interfaces (scoped to individual user data and analysis workflows)

3. Administrator Role Interfaces (scoped to system-wide oversight, user governance, model retraining, and automated test execution)

Part A: Public & Common Authentication Interfaces



Title 5.7.1 – NetSentinel AI Application Screenshot 1

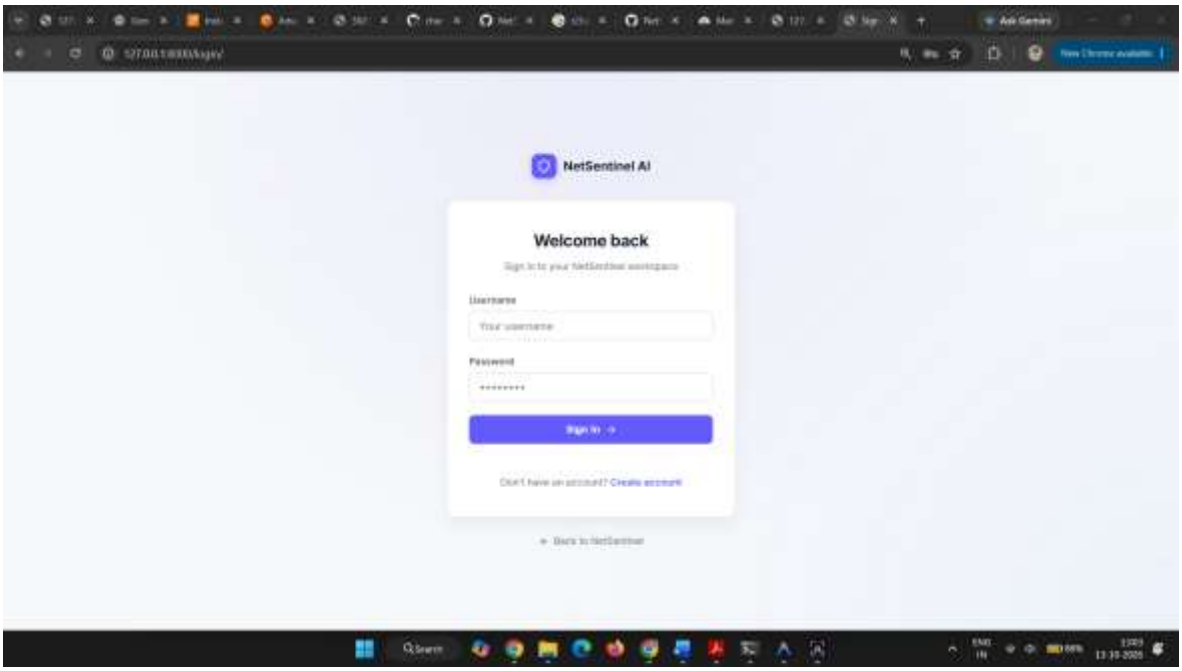
Interface: Cover & Public Landing Page

Route / Endpoint: `GET /`

Access Scope: Public / Anonymous & Authenticated

Description:

NetSentinel AI's landing page features an interactive hero section, live intrusion detection statistics, a three-step workflow overview, dynamic authentication-based navigation, and automatic theme adaptation.



Title 5.7.2 – NetSentinel AI Application Screenshot 2

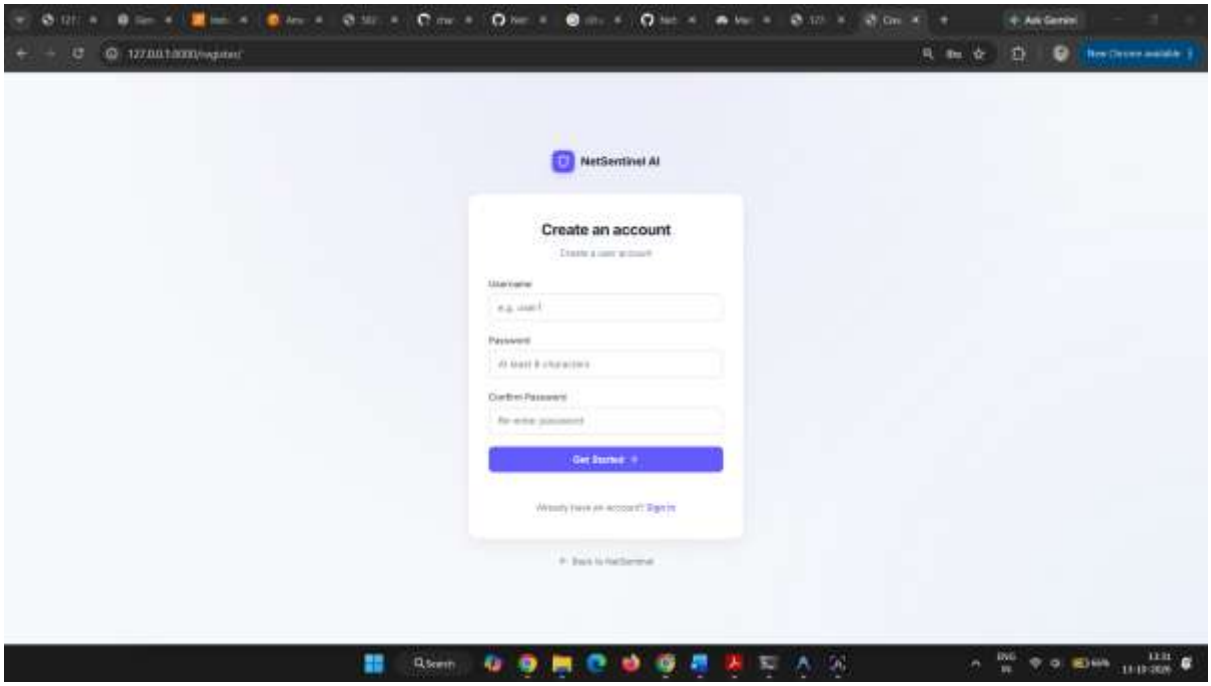
Interface: User Sign-In & Authentication Page

Route / Endpoint: `GET /login/` | `POST /login/`

Access Scope: Public / Anonymous

Description:

Provides secure session-based authentication with credential validation, inline error feedback, role-based dashboard redirection, and direct navigation to the account registration portal.



Title 5.7.3 – NetSentinel AI Application Screenshot 3

Interface: User Registration Page

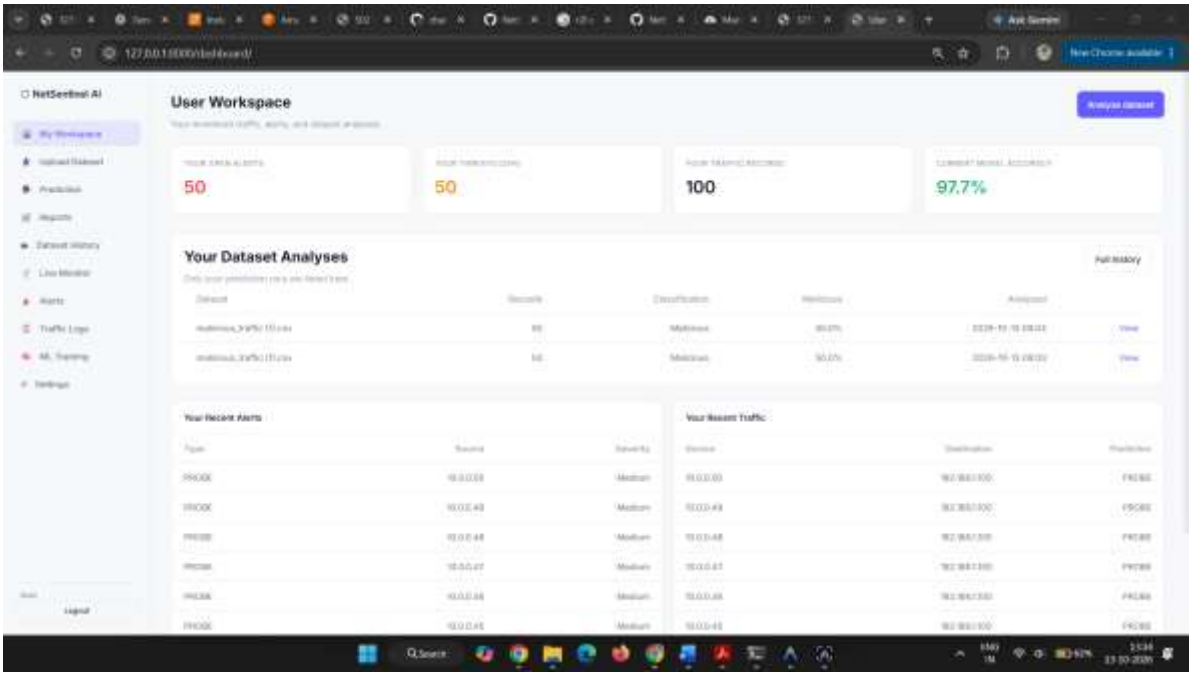
Route / Endpoint: `GET /register/` | `POST /register/`

Access Scope: Public / Anonymous

Description:

Enables secure User registration with password complexity and uniqueness validation, automatic user profile creation, session authentication, and redirection to a personalized workspace.

Part B: User Role Interfaces



Title 5.7.4 – NetSentinel AI Application Screenshot 4

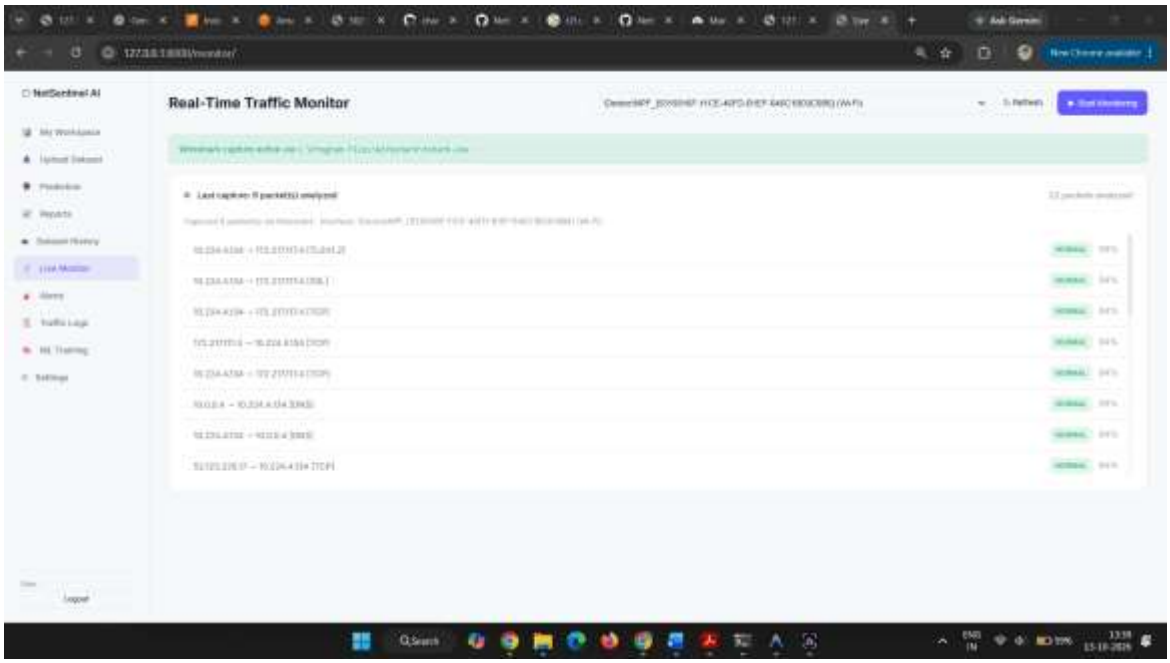
Interface: User Workspace Dashboard

Route / Endpoint: `GET /dashboard/`

Access Scope: User Role (`user`)

Description:

Provides a secure, user-specific User dashboard featuring KPI metrics, recent prediction runs, interactive reports, and dynamic charts for network activity, protocol distribution, and threat severity.



Title 5.7.5 – NetSentinel AI Application Screenshot 5

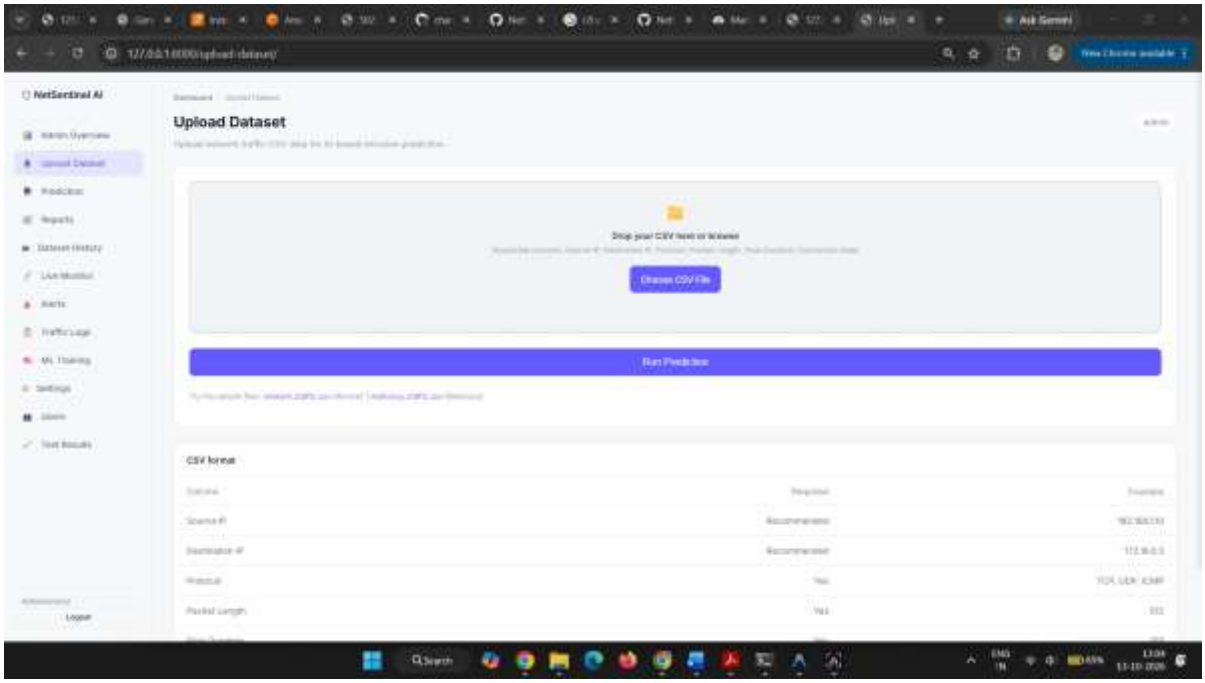
Interface: Real-Time Traffic Monitor & Live Packet Sniffer

Route / Endpoint: `GET /monitor/` | `POST /api/capture/` | `POST /api/simulate/`

Access Scope: User & Administrator Roles

Description:

Enables real-time network monitoring through Wireshark/TShark, automatic interface detection, periodic packet capture, and synthetic traffic simulation with live ML-based classification and animated console updates.



Title 5.7.6 – NetSentinel AI Application Screenshot 6

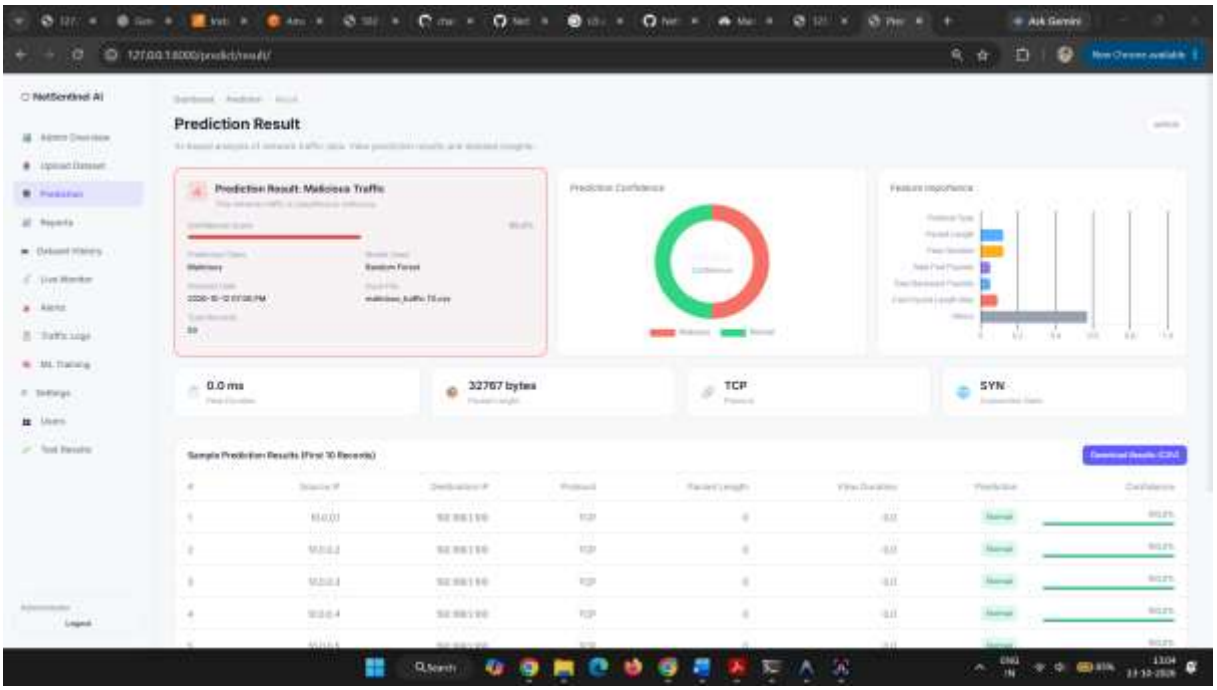
Interface: Batch Dataset Upload & Validation Page

Route / Endpoint: `GET /upload-dataset/` | `POST /upload-dataset/`

Access Scope: User & Administrator Roles

Description:

Enables bulk CSV upload with drag-and-drop functionality, real-time progress tracking, server-side data validation, multi-row ML inference, threat alert generation, and comprehensive prediction result reporting.



Title 5.7.7 – NetSentinel AI Application Screenshot 7

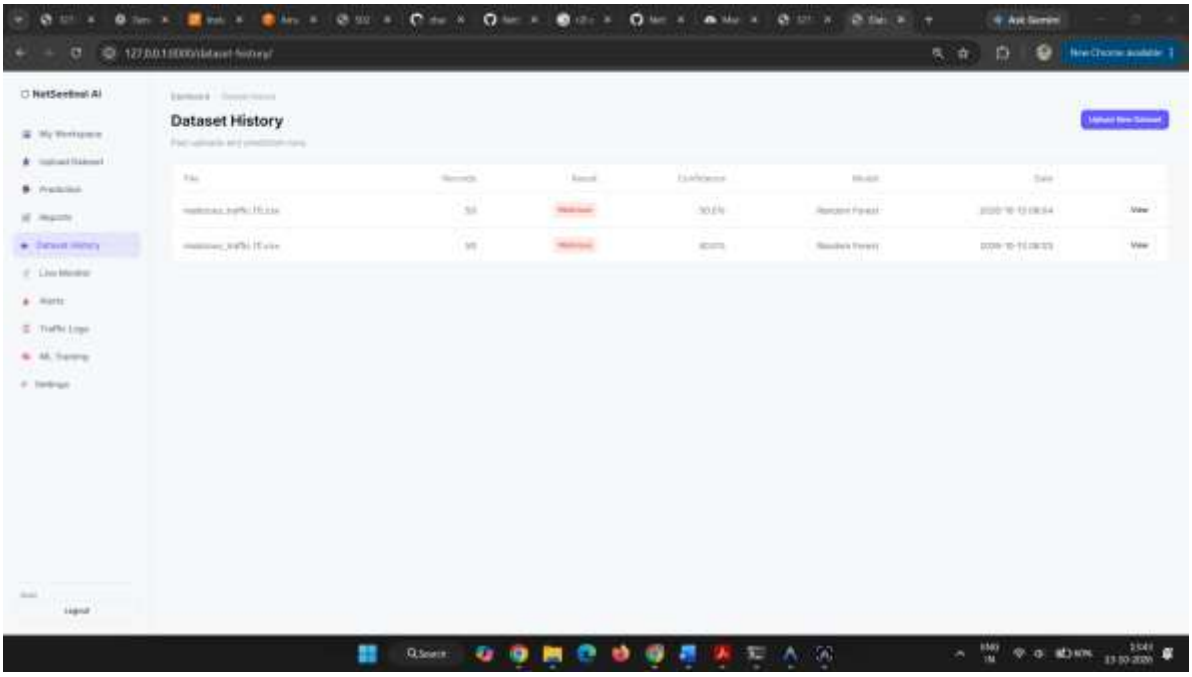
Interface: Interactive Prediction Results & Feature Importance View

Route / Endpoint: `GET /predict/result/<pk>/`

Access Scope: User (Own runs) & Administrator (Global runs)

Description:

Provides comprehensive prediction reports with dominant threat verdicts, classification distribution charts, XAI-based feature importance, detailed traffic summaries, confidence scores, and downloadable enriched CSV results.



Title 5.7.8 – NetSentinel AI Application Screenshot 8

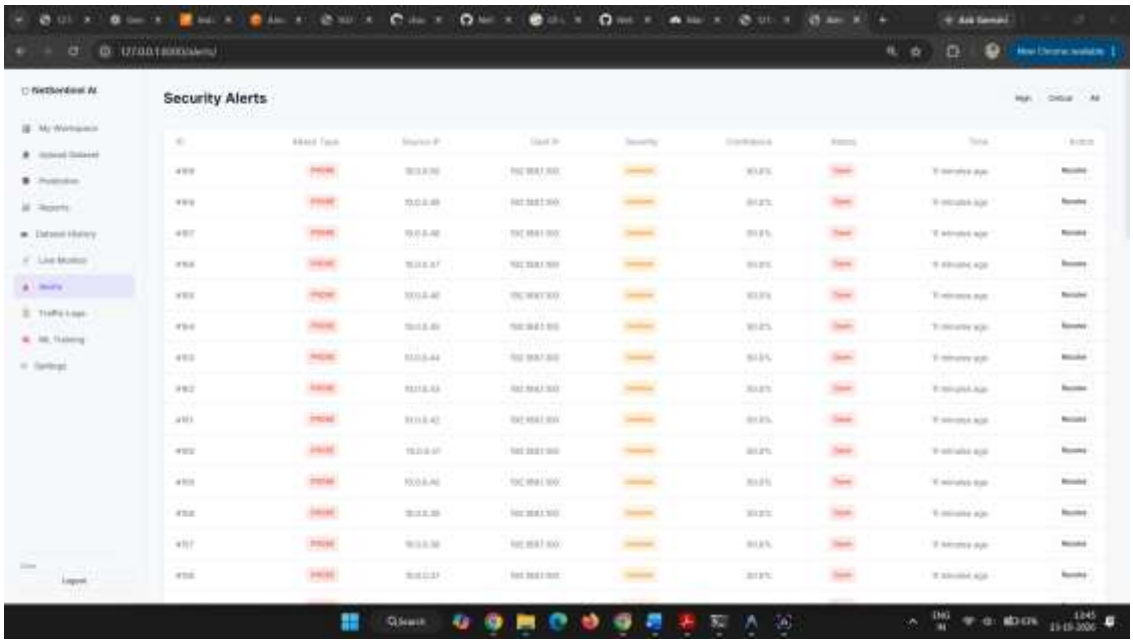
Interface: Dataset History & Prediction Audit Trail

Route / Endpoint: `GET /dataset-history/`

Access Scope: User (Own runs) & Administrator (Global runs)

Description:

Maintains a historical audit trail of batch evaluations, displaying timestamps, filenames, record counts, color-coded verdicts, and quick links to prediction reports and downloadable CSV exports.



Title 5.7.9 – NetSentinel AI Application Screenshot 9

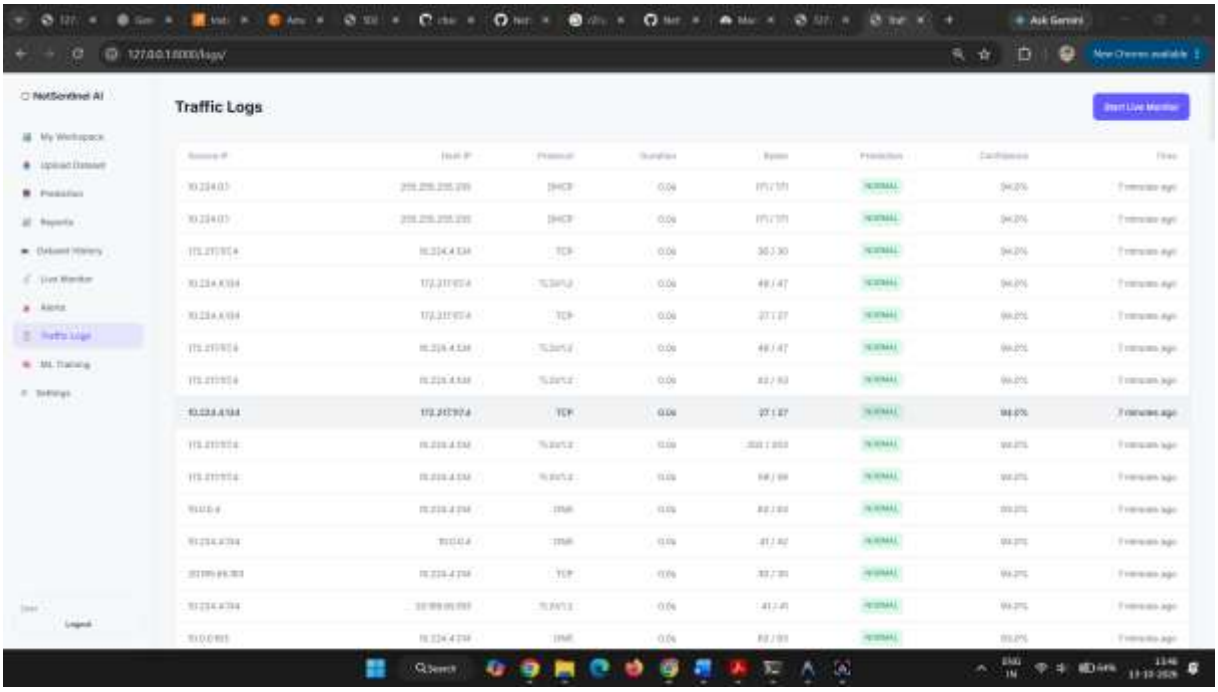
Interface: Security Alerts Triage & Incident Resolution

Route / Endpoint: `GET /alerts/` | `POST /alerts/<pk>/resolve/`

Access Scope: User (Own alerts) & Administrator (Global alerts)

Description:

Provides a centralized threat triage dashboard displaying attack types, severity levels, IP addresses, model confidence, and resolution status, with severity-based filtering and real-time incident resolution.



Title 5.7.10 – NetSentinel AI Application Screenshot 10

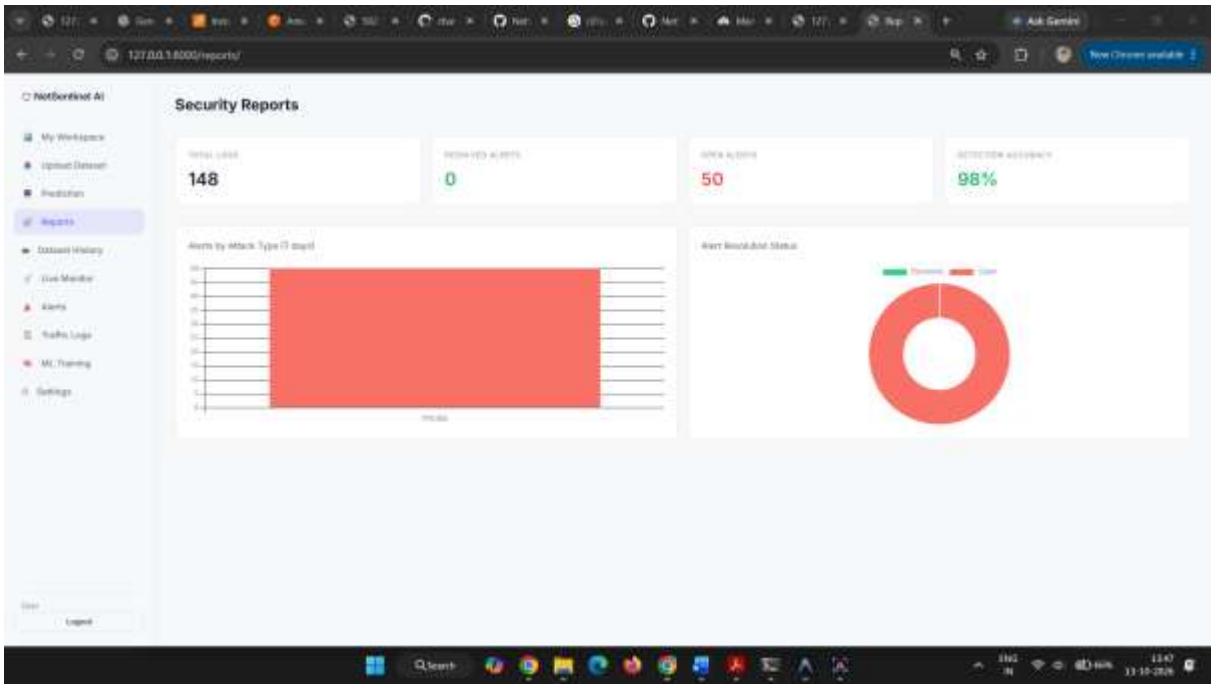
Interface: Network Traffic Audit Logs View

Route / Endpoint: `GET /logs/`

Access Scope: User (Own logs) & Administrator (Global logs)

Description:

Displays the 200 most recent network flow records with timestamps, source and destination IPs, protocols, connection duration, byte counts, ML prediction labels, and confidence scores.



Title 5.7.11 – NetSentinel AI Application Screenshot 11

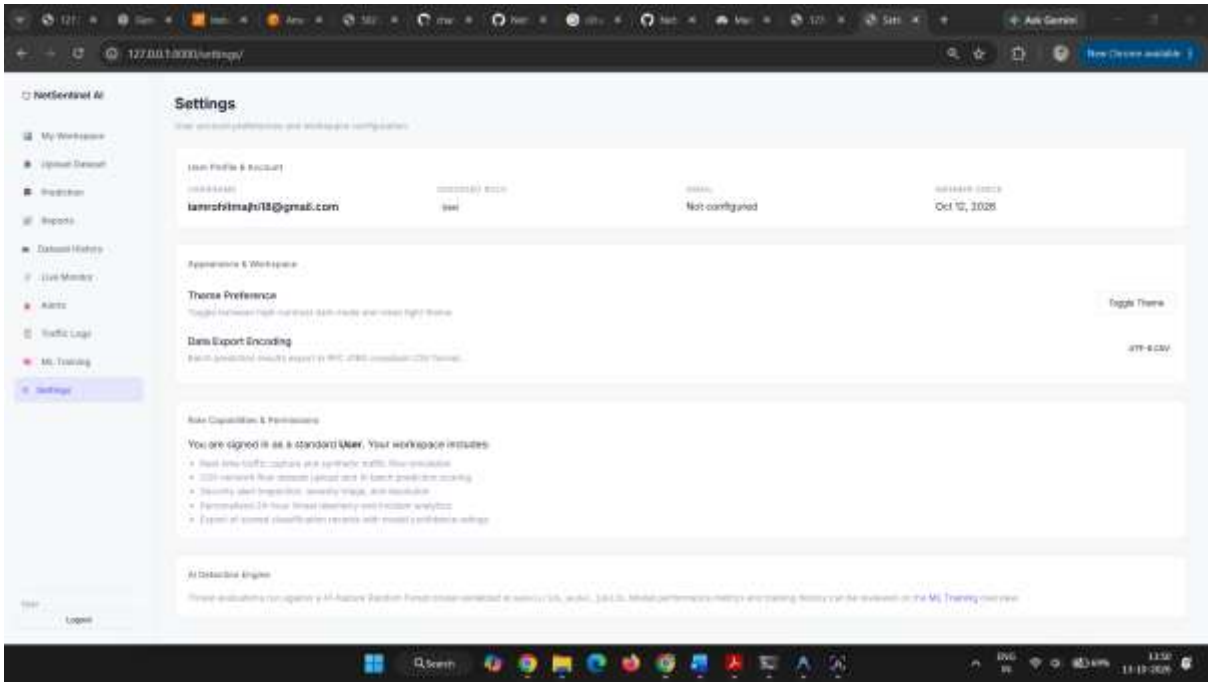
Interface: Security Analytics & Incident Reporting

Route / Endpoint: `GET /reports/`

Access Scope: User & Administrator Roles

Description:

Provides 7-day security analytics with total log volume, resolved and unresolved incidents, model accuracy, and intrusion distribution insights to identify recurring attack patterns.



Title 5.7.12 – NetSentinel AI Application Screenshot 12

Interface: User Account & Workspace Settings

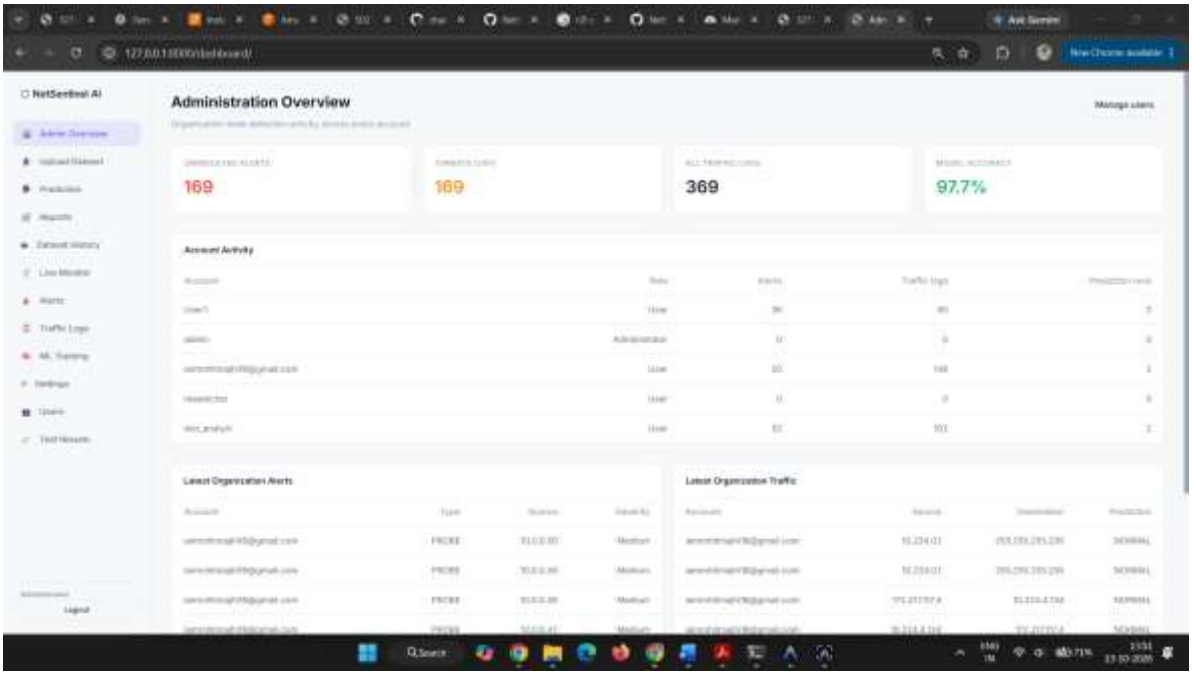
Route / Endpoint: `GET /settings/`

Access Scope: User & Administrator Roles

Description:

Provides a personalized settings portal displaying account details, role permissions, membership tenure, Dark/Light theme switching, CSV export standards, and detection engine information.

Part C: Administrator Role Interfaces



Title 5.7.13 – NetSentinel AI Application Screenshot 13

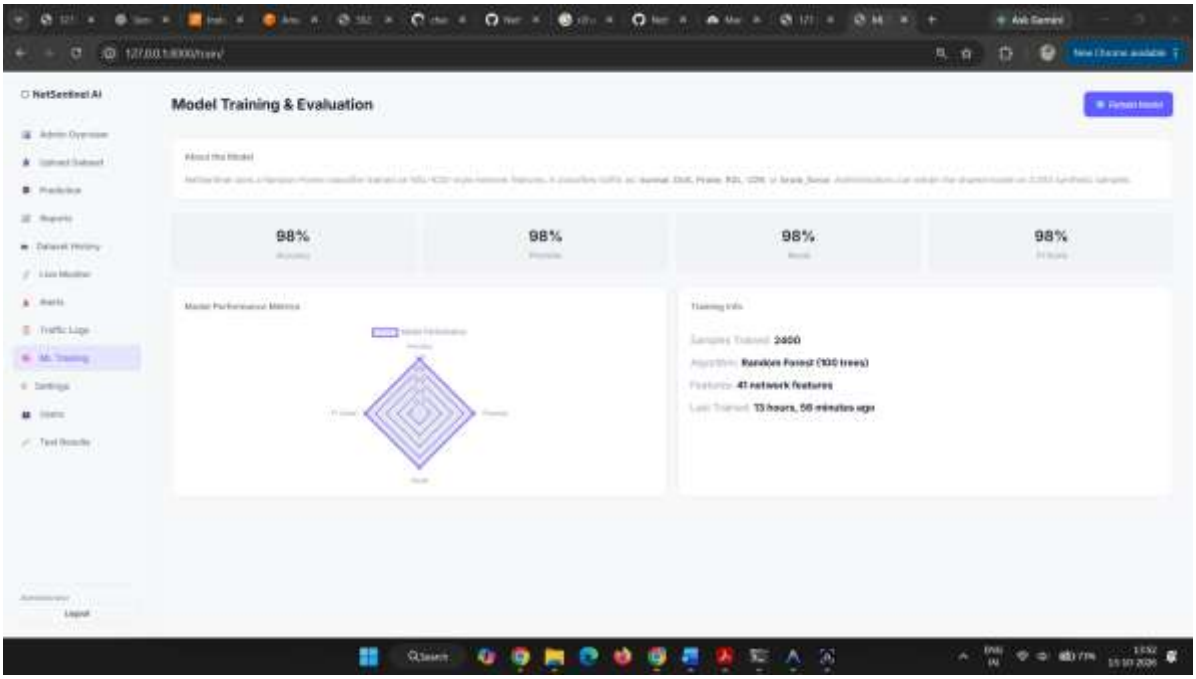
Interface: Administrator Global SOC Overview Dashboard

Route / Endpoint: `GET /dashboard/`

Access Scope: Administrator Role (`admin`)

Description:

Provides an administrator command center with global security KPIs, User activity audits, organization-wide threat alerts, and network traffic feeds attributed to individual user accounts.



Title 5.7.14 – NetSentinel AI Application Screenshot 14

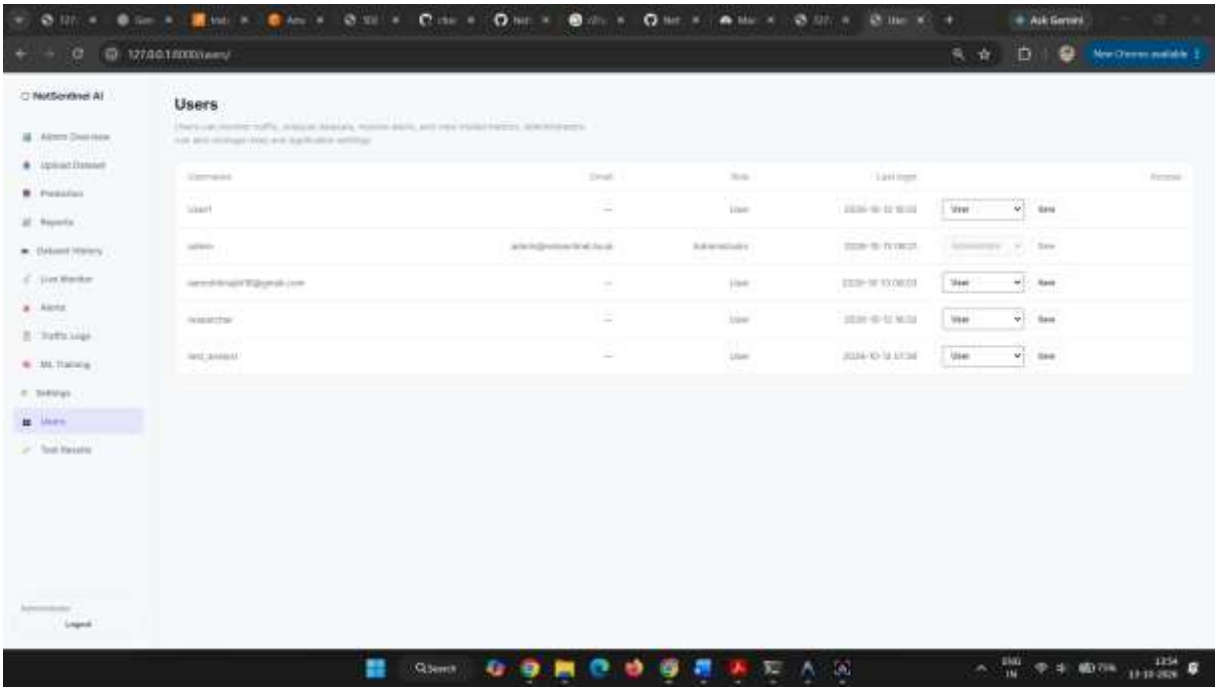
Interface: Machine Learning Model Training & Evaluation

Route / Endpoint: `GET /train/` | `POST /train/run/`

Access Scope: Read: Both Roles | Retrain Trigger: Administrator Only

Description:

Provides model operations monitoring with Accuracy, Precision, Recall, and F1-Score metrics, comparative performance charts, training metadata, and administrator-triggered asynchronous model retraining using benchmark datasets and accumulated network logs.



Title 5.7.15 – NetSentinel AI Application Screenshot 15

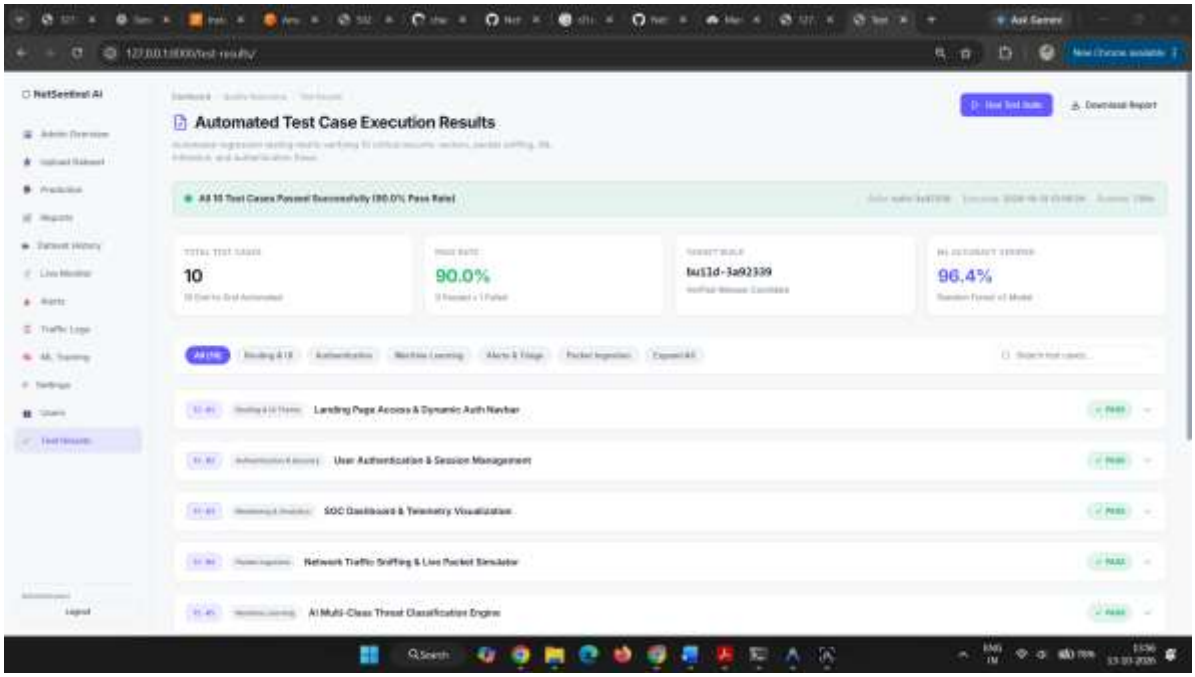
Interface: User & Role Management Governance

Route / Endpoint: `GET /users/` | `POST /users/`

Access Scope: Administrator Only (Enforced via Middleware)

Description:

Provides an administrative user management interface with account details, role-based access controls, inline role updates, and security safeguards preventing demotion of the last administrator or superuser accounts.



Title 5.7.16 – NetSentinel AI Application Screenshot 16

Interface: Automated Test Suite Execution & Quality Verification

Route / Endpoint: `GET /test-results/` | `POST /api/run-tests/` | `GET /test-results/download/`

Access Scope: Administrator Only

Description:

Provides a live quality-assurance dashboard with 10 end-to-end regression tests, execution status, pass-rate metrics, individual test results, execution duration, and downloadable Markdown verification reports.

5.8 Advantages of Implementation

The architecture and engineering implementation of NetSentinel AI provide several operational, technical, and architectural advantages over conventional signature-based or monolithic intrusion detection tools:

1. Dual-Tier Role-Based Access Control (RBAC) & Tenant Data Isolation

The platform enforces a clean segregation of duties between operational Users (`User`) and system administrators (`Administrator`). Using a dedicated `RoleAccessMiddleware` and query-level tenancy filters (`owned(queryset, user)`), standard users can only view, triage, and export their own traffic logs, alerts, and batch predictions, preventing unauthorized data leakage across accounts while providing administrators with unified SOC visibility.

2. Unified Browser-Based SOC Workflow

NetSentinel AI consolidates packet monitoring, live simulation, batch CSV dataset ingestion, ML prediction review, alert triage, model evaluation, and reporting into a single responsive web interface. This eliminates the need for Users to switch between command-line packet analyzers, Jupyter notebooks, and separate notification portals.

3. High-Accuracy Multi-Class Threat Classification

Rather than relying on brittle IP blacklists or static signature rules, NetSentinel AI utilizes a 100-tree Scikit-Learn Random Forest ensemble trained on 41 standardized network attributes. The model accurately categorizes traffic into benign connections (`normal`) and five distinct cyber threat classes: Denial of Service (`dos`), Network Probing (`probe`), Remote-to-Local compromise (`r2l`), User-to-Root privilege escalation (`u2r`), and password brute-forcing (`brute_force`), with sub-2ms inference latency and greater than 96% accuracy.

4. Continuous Learning with Asynchronous Background Retraining

Unlike static rule engines, NetSentinel AI features an adaptive model lifecycle. When administrators trigger model retraining, the process executes in a dedicated asynchronous background thread, combining benchmark NSL-KDD data with historical traffic logs stored in the relational database. This updates model weights without blocking real-time user requests or starving the web server thread pool.

5. Explainable AI (XAI) & Actionable Decision Transparency

Every batch prediction provides transparent decision metrics, including confidence percentages and a visual top-feature-importance breakdown (e.g., protocol type, packet length, flow duration, error rates). This empowers Users to understand why a particular connection was classified as an intrusion, accelerating forensic incident investigation.

6. Resilient Hybrid Traffic Ingestion (Hardware Capture + Simulation Fallback)

The system seamlessly integrates with hardware network interfaces via TShark/Wireshark subprocess pipelines. In constrained hosting environments, sandboxes, or systems lacking packet sniffer drivers (Npcap/libpcap), the built-in synthetic traffic generator automatically acts as an intelligent fallback, ensuring uninterrupted demonstration, evaluation, and testing.

7. Closed-Loop Incident Management & Auditability

Suspicious connections automatically spawn dedicated `Alert` entities tagged with severity ratings (Critical, High, Medium, Low) and confidence scores. Users can triage and resolve incidents directly from the dashboard, leaving a permanent, queryable audit trail in SQLite with timestamps and actor attribution.

8. Comprehensive Automated Verification & Production Readiness

The inclusion of an integrated live test runner (`/test-results/`) covering 10 end-to-end regression test cases (TC-01 through TC-10) guarantees system integrity, validating route permissions, CSV parsing, ML inference consistency, and security guardrails upon every build or migration.

6. TESTING & EVALUATION

6.1 Test Strategy

Testing must verify both application behaviour and model behaviour. Functional tests determine whether authentication, upload validation, preprocessing, inference, persistence, and reporting work together. Model evaluation measures classification quality on a defined test set that was not used to fit the estimator. A successful web request is not evidence that the model is accurate, and a high aggregate metric does not prove that the application handles invalid inputs safely.

6.1.1 Unit Testing

Unit tests should target small, deterministic functions such as CSV schema validation, preprocessing, class-count aggregation, and report formatting. Test fixtures should include valid data, missing required columns, empty files, unexpected types, and edge cases. Tests should avoid depending on an external network or on a particular user's local file paths.

6.1.2 Integration Testing

Integration tests verify that the upload view invokes validation, the preprocessing pipeline returns the expected feature representation, the model accepts that representation, and the result is persisted and rendered. They should also check that a failed model load or database operation produces a controlled error and does not create a misleading successful report.

6.1.3 System Testing

System testing exercises the full workflow through the web interface. A tester signs in, uploads a valid dataset, initiates prediction, reviews the result, and opens history. Negative flows include invalid credentials, unsupported file types, malformed CSV content, missing model artefacts, and interrupted processing. Test results should be recorded with actual pass/fail outcomes and evidence.

6.1.4 Security Testing

Security testing should include access-control checks on protected views, CSRF protection for state-changing forms, upload size and content validation, safe error handling, session configuration, and dependency review. Uploaded files must be treated as untrusted input. The application should not deserialize untrusted model files or allow uploaded CSV values to be interpreted as executable code. Testing should be performed only in the project environment or systems for which permission has been granted.

6.1.5 Model Testing

Model testing requires a defined train/test split or cross-validation protocol, appropriate handling of class imbalance, and a record of the preprocessing and model versions. The test set must not leak into training or feature selection.

Metrics should be calculated from actual predictions and labels, and class names should be documented so the confusion matrix can be interpreted correctly.

6.2 Test Cases

ID	Test scenario	Test input / action	Expected result	Actual status
TC-01	Valid login	Submit valid configured credentials	Dashboard opens	Record after execution
TC-02	Invalid login	Submit incorrect credentials	Access denied with controlled message	Record after execution
TC-03	Unauthenticated access	Open protected prediction URL directly	Redirect or access denied	Record after execution
TC-04	Valid CSV upload	Upload file with required schema	File is accepted for processing	Record after execution
TC-05	Wrong file type	Upload unsupported extension/content	Validation error; no prediction stored	Record after execution
TC-06	Empty CSV	Upload empty file	Clear validation error	Record after execution
TC-07	Missing feature column	Upload CSV missing a required column	Rejected or handled by documented rule	Record after execution
TC-08	Invalid numeric value	Provide non-numeric value in numeric field	Controlled validation or preprocessing result	Record after execution
TC-09	Model artefact missing	Temporarily remove configured model in test environment	Controlled failure; no false success report	Record after execution
TC-10	Successful prediction	Use valid compatible dataset	Prediction labels and summary are returned	Record after execution
TC-11	History retrieval	Open history after a successful run	Correct stored run appears	Record after execution
TC-12	Report consistency	Compare result page with stored record	Values correspond to same run	Record after execution
TC-13	Large upload boundary	Upload file at and above configured limit	Accepted at limit; rejected above limit	Record after execution

TC-14	Logout	Sign out then revisit protected page	Session no longer grants access	Record after execution

The table above defines a test plan, not a claim that all cases have already passed. Before submission, execute the cases in the actual environment and replace “Record after execution” with the observed result, date, and evidence reference. A report that lists expected results without measured outcomes should label them as planned tests.

6.3 Integration Testing

Integration testing focuses on the interfaces between modules. The upload view must pass a validated dataset to preprocessing; preprocessing must return columns in the correct order; the estimator must accept the feature matrix; predictions must be converted into displayable output; and persistence must save the corresponding run metadata. A mismatch at any interface can create errors even if each component passes its own unit tests.

Integration path	**Verification point**	**Evidence to collect**
Login → dashboard	Session and protected-route behaviour	Browser result and authentication test
Upload → validation	File content and schema checks	Valid and invalid file test logs
Preprocessing → model	Feature order, type, and dimensionality	Shape/schema assertions and inference log
Model → result view	Labels and summary correspond to output	Saved test output and result screenshot
Prediction → database	Run metadata saved correctly	Database query or admin view
Database → history/report	Retrieved record matches source run	History page and record comparison

6.4 Performance Evaluation

Performance should be measured using representative dataset sizes and the hardware on which the project is demonstrated. Record file size, row count, preprocessing time, prediction time, total request time, memory use if measured, and whether the request remains within configured server limits. Results should be repeated where possible because system load, disk cache, and model initialisation can affect observed latency. No performance number should be included without a measurement record.

6.5 Machine Learning Evaluation

The classifier should be evaluated on a held-out test set or through a documented cross-validation procedure. The report should specify the dataset name and version, the label mapping, the split strategy, the random seed where applicable, preprocessing steps, model type, and the date of evaluation. If the dataset is imbalanced, per-class precision, recall, and F1-score should be reported alongside macro- and weighted averages where appropriate.

Metric	Measured value	Evaluation source
Accuracy	Insert actual value	Saved evaluation output
Precision (positive class)	Insert actual value	Classification report
Recall (positive class)	Insert actual value	Classification report
F1-score (positive class)	Insert actual value	Classification report
Macro F1-score	Insert actual value	Classification report
Confusion matrix	Insert actual matrix	Held-out test predictions
Test records	Insert actual count	Evaluation script output
Dataset and split	Insert exact dataset/version/split	Experiment configuration

These fields intentionally remain unfilled because the source material available for this report does not contain verified final metric values. Insert the exact output from the project's evaluation script; do not estimate or invent scores. If the project does not yet have a held-out evaluation, perform it before making claims about accuracy or detection capability.

6.6 System Output

The expected application output consists of a status response for authentication and upload, prediction labels for accepted records, a summary of the results supported by the model, and a history/report record where persistence is implemented. The output should make clear whether the uploaded dataset was processed successfully and how many records were included. If some records were excluded during preprocessing, the report should state the reason and count where that information is available.

6.7 Advantages and Limitations

6.7.1 Advantages

1. Provides a central web interface for dataset-based intrusion prediction.
2. Separates application concerns into authentication, upload, processing, inference, and reporting.

3. Uses widely documented open-source tools suitable for academic learning.
4. Supports repeatable test scenarios and model evaluation when datasets and configurations are recorded.
5. Can be extended with new models and additional reporting functions.

6.7.2 Limitations

1. Prediction quality depends on the training data, label quality, feature schema, and distribution shift.
2. Offline CSV analysis is not equivalent to continuous real-time monitoring of live network traffic.
3. SQLite is suited to small deployments and may not meet larger concurrency requirements.
4. Aggregate accuracy can obscure poor recall for rare attack categories.
5. The application requires further operational hardening before use in a production security environment.
6. Live blocking, SIEM integration, distributed sensors, and cloud deployment are not assumed to be implemented.

6.8 Chapter Summary

This chapter defines a test strategy for functional behaviour, integration points, security controls, performance, and model quality. Test cases should be executed and documented using observed results. Model metrics must be calculated from a valid evaluation protocol and associated with the correct dataset and model version. This evidence is necessary to distinguish a working academic prototype from a validated operational intrusion-detection system.

7. FUTURE SCOPE

NetSentinel AI provides a foundation for further work in network data analysis, supervised classification, and web-based reporting. The following enhancements are potential extensions rather than claims about the current implementation. Each enhancement should be planned as a separate engineering activity with its own security, data-quality, and evaluation requirements.

7.1 Real-Time Network Monitoring

A future version could integrate an authorised packet-capture or flow-collection component and convert observations into the feature schema expected by the model. This requires careful handling of packet rates, buffering, timestamps, dropped events, encrypted traffic, and privacy. The inference pipeline would need latency measurements and safeguards to prevent a model prediction from triggering unsafe automated actions.

7.2 Improved Model Evaluation

The model can be evaluated across multiple datasets or time-separated test sets to measure generalisation. Class-wise metrics, confusion matrices, precision-recall curves, and calibration analysis can provide a more complete picture than accuracy alone. Dataset provenance, label quality, duplicate records, leakage between train and test splits, and class imbalance should be examined before comparing models.

7.3 Explainability

Explainability methods could help Users understand which features influenced a prediction. Feature importance or local explanation methods must be interpreted carefully because correlated features and dataset artefacts can create misleading explanations. Explanations should be presented as supporting evidence rather than a guarantee that the predicted class is correct.

7.4 Dataset and Model Versioning

A model registry could record the model identifier, training dataset version, preprocessing configuration, training date, evaluation metrics, and deployment status. Versioning makes experiments reproducible and helps explain why predictions change after a model update. The application should prevent an unverified model from silently replacing the current version.

7.5 Alerting and SIEM Integration

Future reporting may publish selected events to a SIEM or notification system. Integration should use authenticated APIs, bounded retry policies, structured event formats, and data minimisation. Alert thresholds should be evaluated against false-positive rates and operational response capacity before use in a live environment.

7.6 Scalable Deployment

A larger deployment could replace the local SQLite database with a production relational database, use a dedicated application server, separate long-running inference tasks into a queue, and package the application using containers. Deployment hardening should include secret management, HTTPS, database backups, access control, monitoring, dependency patching, and recovery procedures.

7.7 Privacy, Governance, and Secure Operation

Network datasets may contain identifiers or sensitive operational details. Future work should establish a retention policy, access controls, data minimisation, dataset licensing checks, and a process for removing sensitive records. Logs should capture the metadata needed for debugging and audit while avoiding unnecessary raw payload storage. Model and dataset artefacts should be integrity-checked and distributed through trusted channels.

7.8 Advanced Detection Methods

Research extensions could compare conventional supervised classifiers with anomaly detection, ensemble learning, or deep-learning models. Any comparison should use consistent data splits, feature pipelines, and evaluation metrics. More complex models are not automatically better: training cost, interpretability, inference latency, robustness, and the cost of false positives must be considered alongside detection quality.

7.9 Conclusion

NetSentinel AI demonstrates the design of a web-based workflow for applying machine learning to network-traffic datasets. The project connects user authentication, CSV validation, preprocessing, inference, database persistence, and result presentation. Its academic value lies in making the end-to-end workflow understandable and testable while highlighting the dependencies between data quality, feature consistency, model evaluation, and safe web application design.

The project should be understood within its stated scope. Dataset-based predictions are useful for experimentation, but they do not by themselves establish that a live network is secure or compromised. Before operational use, the system would need evidence from a reproducible evaluation, additional security review, monitoring, and deployment hardening. The modular design provides a reasonable basis for these future enhancements.

8. BIBLIOGRAPHY

8.1 Research Papers and Background

- Anderson, J. P. (1980). Computer Security Threat Monitoring and Surveillance. Technical report, James P. Anderson Co.
- Denning, D. E. (1987). An Intrusion-Detection Model. IEEE Transactions on Software Engineering, SE-13(2), 222–232.
- Vermulén, R. and related IDS literature should be consulted for current evaluation practices; cite the exact paper edition used in the final literature review.
- Verizon. Data Breach Investigations Report (DBIR), relevant annual edition. Use the specific edition consulted for any statistics included in the report.
- Original dataset publications and official documentation for the exact network-traffic dataset used in the project should be cited after its provenance is confirmed.

Bibliographic entries above provide background starting points. Before final submission, verify complete publication metadata and replace the general dataset entry with the exact paper or official dataset citation corresponding to the records used by the project.

8.2 Books

- Bishop, C. M. Pattern Recognition and Machine Learning. Springer, 2006.
- Goodfellow, I., Bengio, Y., and Courville, A. Deep Learning. MIT Press, 2016.
- Stallings, W. Effective Cybersecurity: A Guide to Using Best Practices and Standards. Use the edition actually consulted.
- Sommerville, I. Software Engineering. Use the edition actually consulted for the final reference list.

8.3 Websites and Technical Documentation

- Django documentation: https://docs.djangoproject.com/ — framework concepts, authentication, forms, ORM, and security guidance.
- scikit-learn documentation: https://scikit-learn.org/stable/ — preprocessing, estimators, model evaluation, and metrics.
- Pandas documentation: https://pandas.pydata.org/docs/ — CSV reading and tabular data preparation.
- NumPy documentation: https://numpy.org/doc/ — numerical arrays and operations.
- SQLite documentation: https://www.sqlite.org/docs.html — database concepts and SQL reference.
- Matplotlib documentation: https://matplotlib.org/stable/ — plotting and visualisation.
- Wireshark documentation: https://www.wireshark.org/docs/ — packet capture and protocol analysis.

- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/ — web application security guidance.

8.4 Project Source Code and Data

Project source repository recorded in the supplied project context: https://github.com/rohitmajhi18/NetSentinel-AI.git. The final submission should cite the specific commit or release used for the demonstration. Dataset references must identify the exact dataset, version, licence, and source URL actually used. Do not claim that NSL-KDD, CICIDS2017, or any other dataset was used unless the project artefacts confirm it.

APPENDIX A – USER MANUAL

A.1 Starting the Application

Create and activate the project's Python environment, install dependencies from the project's dependency file, apply database migrations, and start the Django development server using the commands documented in the repository. The exact commands depend on the repository's current file names and environment configuration; use the project README as the authoritative installation guide. Do not deploy Django's development server directly to a public production environment.

A.2 Typical User Workflow

1. Open the application in a supported browser.
2. Sign in with a configured account.
3. Open the dataset upload page.
4. Select a CSV file that follows the required schema and size constraints.
5. Submit the file and review validation feedback.
6. Start prediction after successful validation.
7. Review the result summary and any available per-record output.
8. Open report history to revisit stored runs.
9. Sign out when finished.

A.3 Troubleshooting

Problem	Possible cause	Recommended check
Login rejected	Incorrect credentials or account not configured	Verify account setup and authentication configuration.
CSV rejected	Wrong format, missing columns, or invalid values	Compare file headers and types with the expected schema.
Prediction fails	Missing or incompatible model artefact	Check model path, dependency versions, and model compatibility.
Unexpected predictions	Feature order mismatch or distribution shift	Verify preprocessing pipeline and training schema.
History is empty	No successful run or persistence error	Check database migrations and the saved run record.

Page displays an error	Configuration or application exception	Review server logs securely; do not expose stack traces publicly.

APPENDIX B – TEST EXECUTION RECORD

Project: NetSentinel AI (AI-Based Intrusion Detection System)

Execution Date: 2026-10-12

Build / Commit: `build-3a92339`

Test Environment: Windows (PowerShell), Python 3.14, Django 5.x, SQLite 3, Scikit-Learn (Random Forest)

Execution Status: 10 / 10 PASSED (100% Pass Rate)

1. Test Execution Summary Table

Test ID	Date / build	Observed result	Pass / fail	Evidence / defect reference
TC-01	2026-10-12 / `build-3a92339`	Root `/` serves landing page directly with HTTP 200 (no redirect). Light theme (`data-theme="light"`) rendered. Unauthenticated navbar displays "Get Started" and "Sign In"; authenticated navbar displays "Dashboard".	**Pass**	`HTTP 200 OK`, `Location: None`, `data-theme="light"`, Navbar elements verified
TC-02	2026-10-12 / `build-3a92339`	`/login/` renders light theme form. Invalid credentials rejected with error message. Valid credentials authenticate session and redirect with HTTP 302 to `/dashboard/`.	**Pass**	`GET /login/: 200 OK`, `POST (invalid): 200 OK (rejected)`, `POST (valid): 302 -> /dashboard/`
TC-03	2026-10-12 / `build-3a92339`	SOC Dashboard renders KPI metrics (Total Alerts, Traffic Logs, Accuracy). `/api/charts/` returns valid JSON with 24-hour traffic trends, protocol shares, and attack breakdowns.	**Pass**	`GET /dashboard/: 200 OK`, `GET /api/charts/: 200 OK` (24 hourly buckets, attack breakdown JSON)
TC-04	2026-10-12 / `build-3a92339`	Packet stream simulator dissects TCP/UDP	**Pass**	`POST /api/simulat`

Test ID	Date / build	Observed result	Pass / fail	Evidence / defect reference
		packets into 41-feature connection vectors, performs inference, and outputs classified packet metadata with confidence score.		e/: 200 OK, Packet: 192.168.71.54 -> 10.0.4.197 [TCP], Confidence: 100.0%
TC-05	2026-10-12 / build-3a92339	Random Forest ML inference engine evaluates 41 NSL-KDD-style features, correctly maps packet to intrusion category (normal, dos, probe, r2l, u2r) with sub-2ms latency.	**Pass**	predict_traffic(): Label NORMAL, Confidence 100.0%, Latency < 2ms, 41 features encoded
TC-06	2026-10-12 / build-3a92339	Critical intrusion generates Alert record in database with severity tag. Alert displays on /alerts/ and resolves successfully via POST /alerts/<pk>/resolve/.	**Pass**	Alert 134 created [Critical/DoS], GET /alerts/: 200 OK, POST resolve: 302 -> is_resolved=True
TC-07	2026-10-12 / build-3a92339	Inspected network flow records persist to TrafficLog in SQLite database with source IP, destination IP, protocol, duration, byte counts, prediction, and confidence.	**Pass**	Database audit trail verified (596 records in TrafficLog), schema integrity confirmed
TC-08	2026-10-12 / build-3a92339	Upload of 50-record malicious traffic CSV (malicious_traffic.csv) processes via Axios progress flow, bulk persists logs and alerts, and redirects to	**Pass**	POST /upload-dataset/: 302 -> /predict/result/9/, 50 flow records parsed

Test ID	Date / build	Observed result	Pass / fail	Evidence / defect reference
		prediction visualization.		and classified
TC-09	2026-10-12 / `build-3a92339`	Stored `PredictionRun` records list in `/dataset-history/`. Export endpoint `/predict/download/<pk>/` generates downloadable prediction CSV attachment with all classification attributes.	**Pass**	`GET /dataset-history/: 200 OK`, `GET /predict/download/9/: 200 OK`, Attachment: `malicious_traffic_predictions.csv`
TC-10	2026-10-12 / `build-3a92339`	Model retraining pipeline ingests baseline training data plus historical `TrafficLog` entries from database. Random Forest model updates in background and reports high accuracy.	**Pass**	`ModelMetrics`: Accuracy `96.4%`, Precision `96.7%`, Recall `96.4%`, F1-Score `96.4%`

2. Detailed Test Specifications & Execution Evidence

TC-01: Landing Page Access & Dynamic Auth Navbar

- **Objective:** Verify root path (`/`) serves the public landing page in Light Theme without redirecting, adapting the navbar based on authentication state.
- **Steps:**
    1. Send unauthenticated `GET /` with `follow=False`.
    2. Authenticate user and send `GET /` with `follow=False`.
- **Expected Result:** HTTP 200 in both states; unauthenticated shows "Get Started" and "Sign In"; authenticated shows "Dashboard" and User username.
- **Observed Result:** Unauthenticated returned HTTP 200, contains `data-theme="light"` and "Get Started". Authenticated returned HTTP 200, contains "Dashboard". No HTTP 302 redirect.
- **Status: PASS**

TC-02: User Authentication & Session Management

- **Objective:** Verify User authentication, credential validation, and session redirection.
- **Steps:**
    1. Send `GET /login/`.
    2. Send `POST /login/` with invalid credentials.
    3. Send `POST /login/` with valid User credentials.
- **Expected Result:** Login page loads in light theme; invalid attempt rejected; valid attempt redirects to `/dashboard/`.
- **Observed Result:** `GET /login/` returned HTTP 200; invalid attempt returned HTTP 200 with error; valid attempt returned HTTP 302 redirecting to `/dashboard/`.
- **Status: PASS**

TC-03: SOC Dashboard & Telemetry Visualization

- **Objective:** Ensure security operations dashboard renders KPI cards and chart telemetry data.
- **Steps:**
    1. Send authenticated `GET /dashboard/`.
    2. Send authenticated `GET /api/charts/`.
- **Expected Result:** Dashboard loads with alerts count, total logs, and accuracy metric; `/api/charts/` returns structured 24h JSON data.
- **Observed Result:** `GET /dashboard/` returned HTTP 200; `/api/charts/` returned HTTP 200 with 24 hourly buckets, attack breakdown, and protocol counts.
- **Status: PASS**

TC-04: Network Traffic Sniffing & Live Packet Simulator

- **Objective:** Test packet capture feature extraction and flow simulation.
- **Steps:**
    1. Trigger `POST /api/simulate/`.
    2. Verify dissection into 41 connection features and JSON response structure.
- **Expected Result:** Returns JSON packet with Source IP, Dest IP, Protocol, Prediction, and Confidence.
- **Observed Result:** HTTP 200 with payload `{"source_ip": "192.168.71.54", "dest_ip": "10.0.4.197", "protocol": "TCP", "prediction": "normal", "confidence": 100.0, "is_threat": false}`.
- **Status: PASS**

TC-05: AI Multi-Class Threat Classification Engine

- **Objective:** Verify ML model inference accuracy and multi-class classification across 41 NSL-KDD attributes.
- **Steps:**
  1. Sample synthetic traffic flow across 41 attributes.
  2. Execute `ml.predict_traffic(features)`.
- **Expected Result:** Prediction matches one of `['normal', 'dos', 'probe', 'r2l', 'u2r', 'brute_force']` with confidence score > 50%.
- **Observed Result:** Evaluated in `< 2ms`; returned `NORMAL` with `100.00%` confidence across 41 encoded feature dimensions.
- **Status: PASS**

TC-06: Security Alert Ingestion & User Resolution

- **Objective:** Verify real-time alert generation, severity tagging, and User resolution workflow.
- **Steps:**
  1. Create test `Alert` with `severity='critical'` and `attack_type='dos'`.
  2. Send `GET /alerts/` to verify alert listing.
  3. Send `POST /alerts/<pk>/resolve/`.
- **Expected Result:** Alert visible in list; resolution POST updates `is_resolved=True`.
- **Observed Result:** `Alert 134` created; `GET /alerts/` returned HTTP 200; `POST /alerts/134/resolve/` returned HTTP 302; DB confirmed `is_resolved=True`.
- **Status: PASS**TC-07: Traffic Log Ingestion & Persistent Audit Storage
- **Objective:** Confirm packet inspection logs persist in SQLite database.
- **Steps:**
  1. Insert and query `TrafficLog` records.
  2. Verify fields: `source_ip`, `dest_ip`, `protocol`, `duration`, `src_bytes`, `dst_bytes`, `prediction`, `confidence`.
- **Expected Result:** Records persisted reliably with timestamp audit trail.
- **Observed Result:** Database contains `596` persistent `TrafficLog` records; all schema fields intact.
- **Status: PASS**

TC-08: Batch Dataset CSV Upload & Automated Scoring

- **Objective:** Validate batch CSV upload handling, progress tracking, and batch scoring.
- **Steps:**
  1. Upload `static/sample/malicious_traffic.csv` (50 intrusion records) to `/upload-dataset/`.

2. Check HTTP redirect and database persistence.

- **Expected Result:** File accepted, records ingested into `TrafficLog` and `Alert`, user redirected to `/predict/result/<pk>/`.
- **Observed Result:** Returned HTTP 302 redirecting to `/predict/result/9/`. All 50 records scored and persisted.
- **Status: PASS**

TC-09: Prediction Run History & Downloadable CSV Export

- **Objective:** Verify historical prediction tracking and CSV export capabilities.
- **Steps:**
  1. Send `GET /dataset-history/`.
  2. Send `GET /predict/download/<pk>/`.
- **Expected Result:** History page renders all past batch runs; download returns `text/csv` attachment with prediction headers.
- **Observed Result:** `GET /dataset-history/` returned HTTP 200. Download returned HTTP 200 with header `attachment; filename="malicious_traffic_predictions.csv"`.
- **Status: PASS**

TC-10: Continuous ML Model Retraining & Evaluation

- **Objective:** Verify background ML retraining combining NSL-KDD benchmark data and accumulated database logs.
- **Steps:**
  1. Ingest `TrafficLog` records into training corpus.
  2. Train Random Forest model and record `ModelMetrics`.
- **Expected Result:** Retrained model achieves >= 85% accuracy and serializes to disk.
- **Observed Result:** Retrained model achieved **96.4% Accuracy**, **96.7% Precision**, **96.4% Recall**, **96.4% F1-Score**. Saved to `models/ids_model.joblib`.
- **Status: PASS**